AccuKnox

Secure Code to Cognition™

ASPM Playbook

Cut through 10,000 findings. Fix the five that can actually bring you down.

Certified & Accredited by

NVIDIA Inception Program · AWS Partner Containers Software · NetApp Excellerator · Open Horizon · NASSCOM Emerge 50 Awards · Nutanix

As Featured In

Gartner · IBM Cloud · Oracle · aws · LF Edge · Nephio

Available On

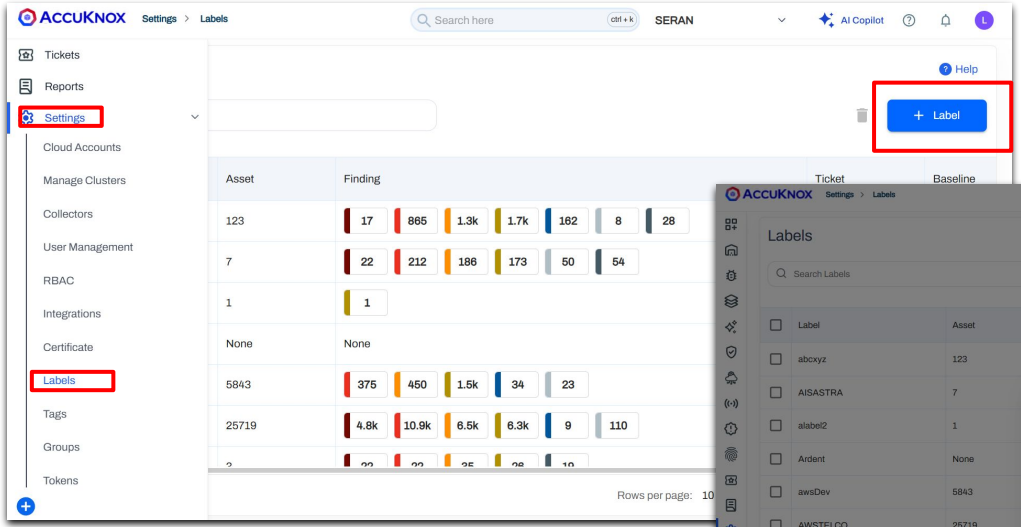
awsmarketplace · Microsoft Azure Marketplace · Red Hat Marketplace · Oracle Cloud Marketplace

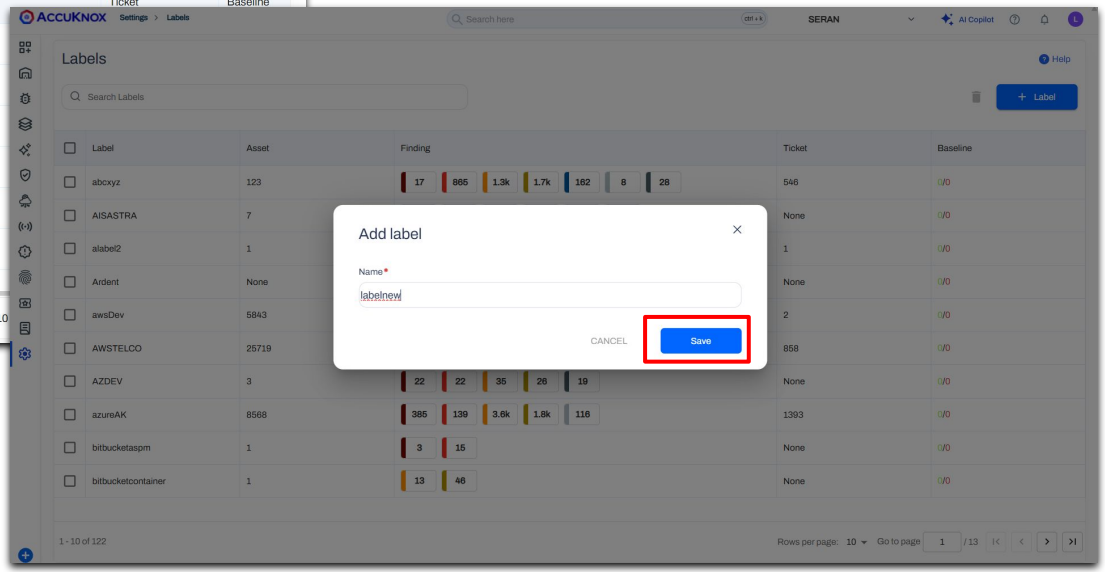
Table Of Contents

Click here to explore the Help Docs for full documentation for various modules.

Generate AccuKnox Label for CI/CD pipeline

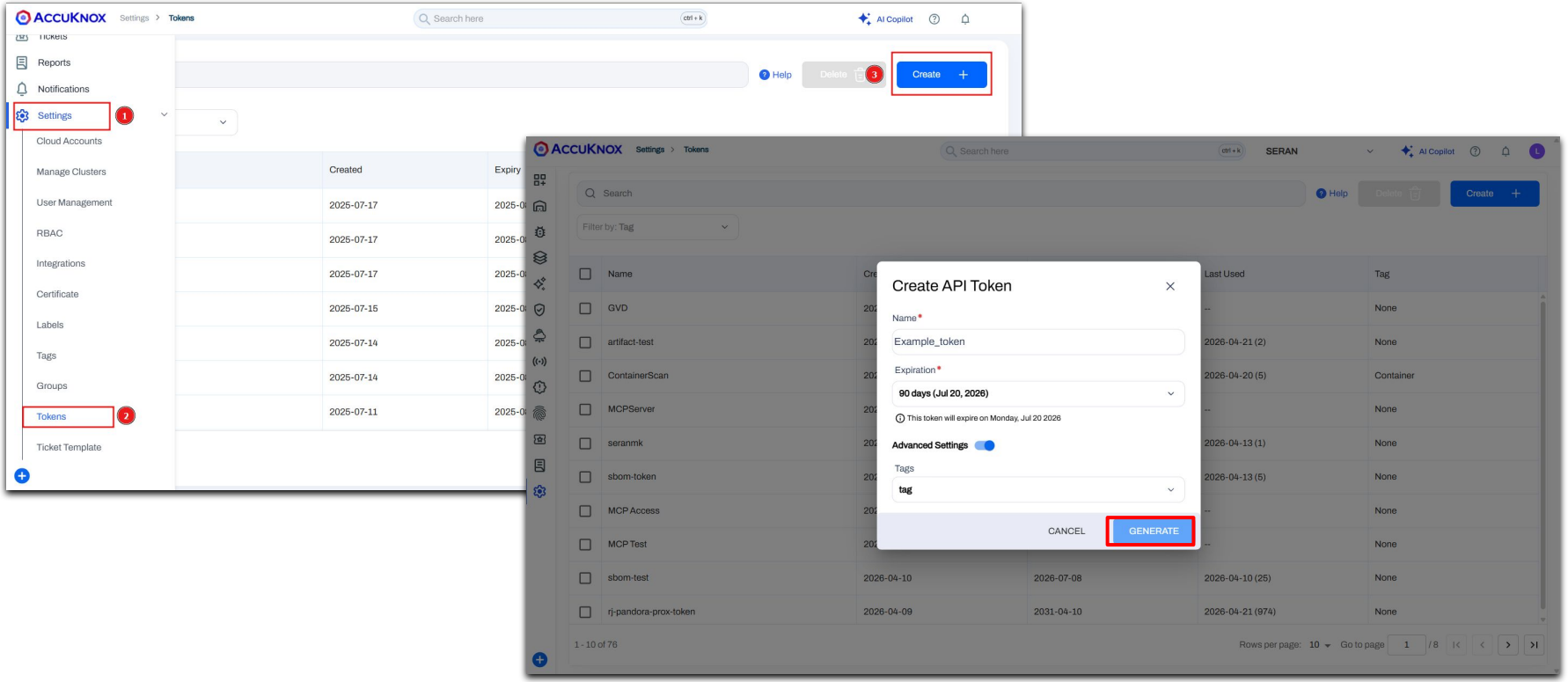
- To generate a label, open AccuKnox and navigate to **Settings > Labels > Create.**
- Copy the label, then configure them as secrets in your CI/CD pipeline.





Generate AccuKnox API token for CI/CD pipeline

- To generate a token, open AccuKnox and navigate to **Settings > Tokens > Create.**
- Generate a token, and copy the token, then configure them as secrets in your CI/CD



AccuKnox

Container Scan

ASPM Integration

Configuring the Container Scan in GitHub actions

Step 1: Add AccuKnox Container Scan to Your GitHub Workflow

- Open your GitHub repository and navigate to your workflow file (typically .github/workflows/your-workflow.yml).
- After the build step, add the AccuKnox container scan GitHub Action.
- Copy the label and token created, then configure them as secrets in your CI/CD along with Accuknox Endpoint.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select Container Image Findings.
- Click on any finding that interests you to view detailed information and recommendations.

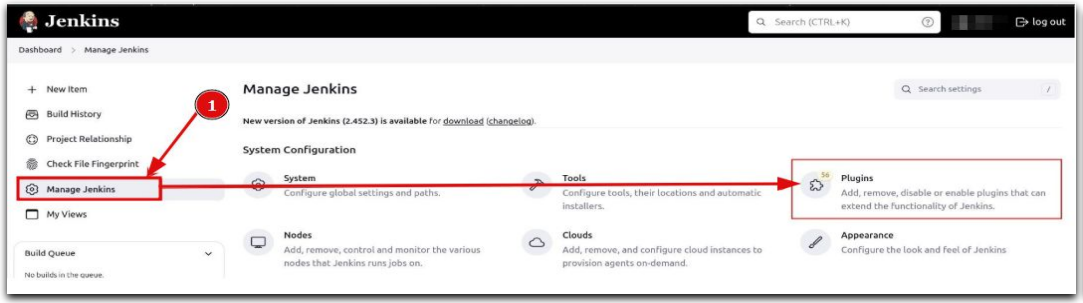
Visit Help Docs for complete guidance

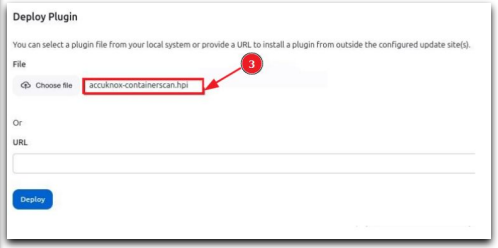
```yaml
jobs:
  accuknox-cicd:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@main

      - name: Build Docker image
        run: |
          docker buildx build .

      - name: AccuKnox Container Scan
        uses: accuknox/container-scan-action@v0.0.1
        with:
          token: ${{ secrets.TOKEN }}
          tenant_id: ${{ secrets.TENANT_ID }}
```

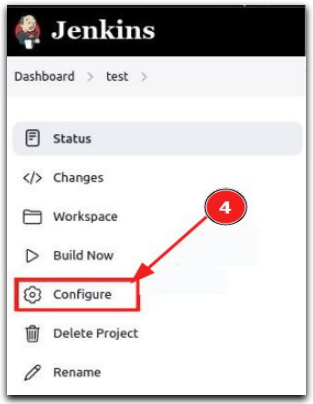
Configuring the Container Scan in Jenkins

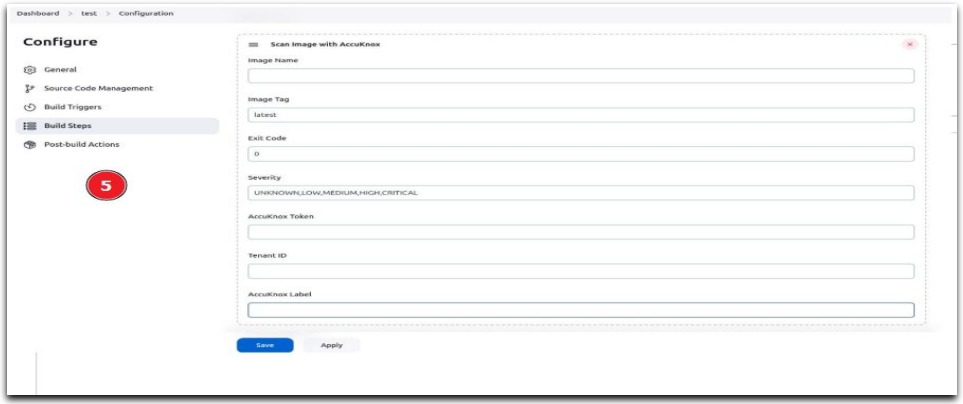
ACCUKNOX

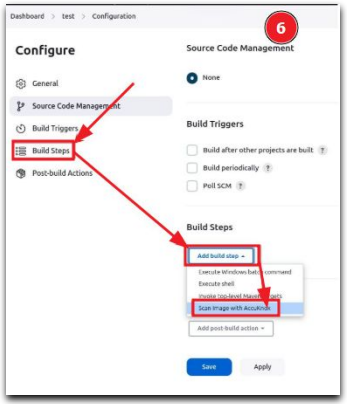
- Manage Jenkins > Plugins > Advanced settings > Upload & install
- Open your Jenkins job, add build step, select ***Scan image with AccuKnox***, and fill required details







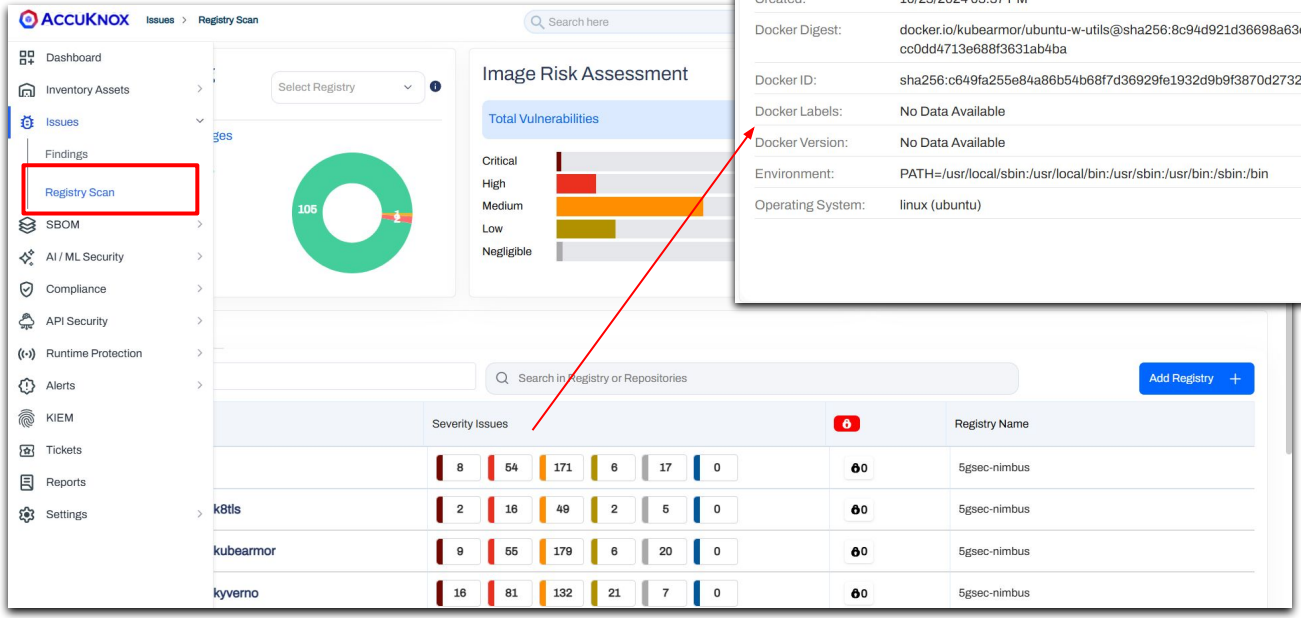


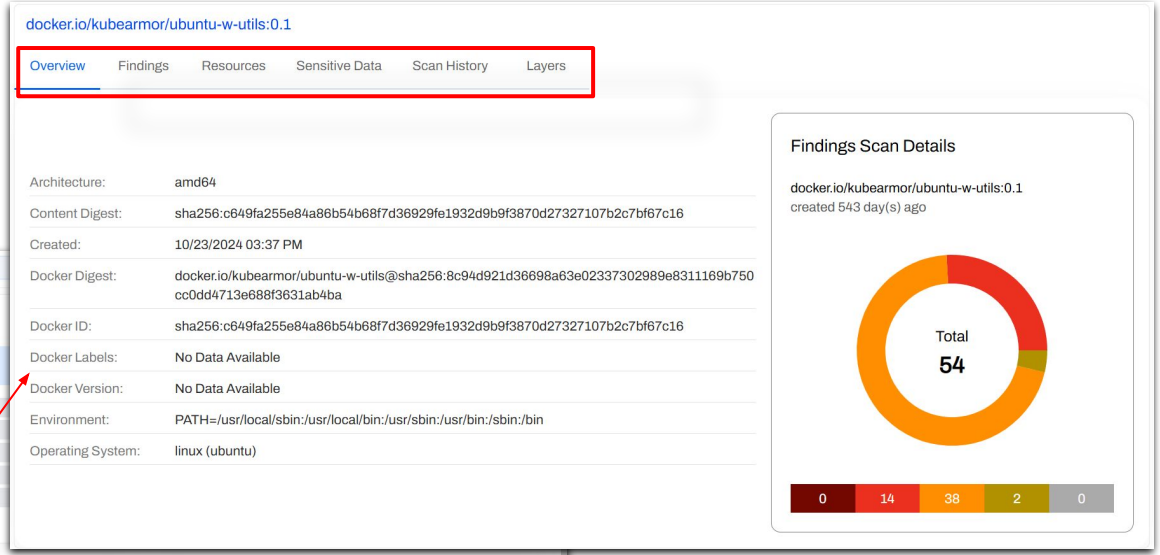


[Visit Help Docs for complete guidance]

View Findings in Registry Scan Page

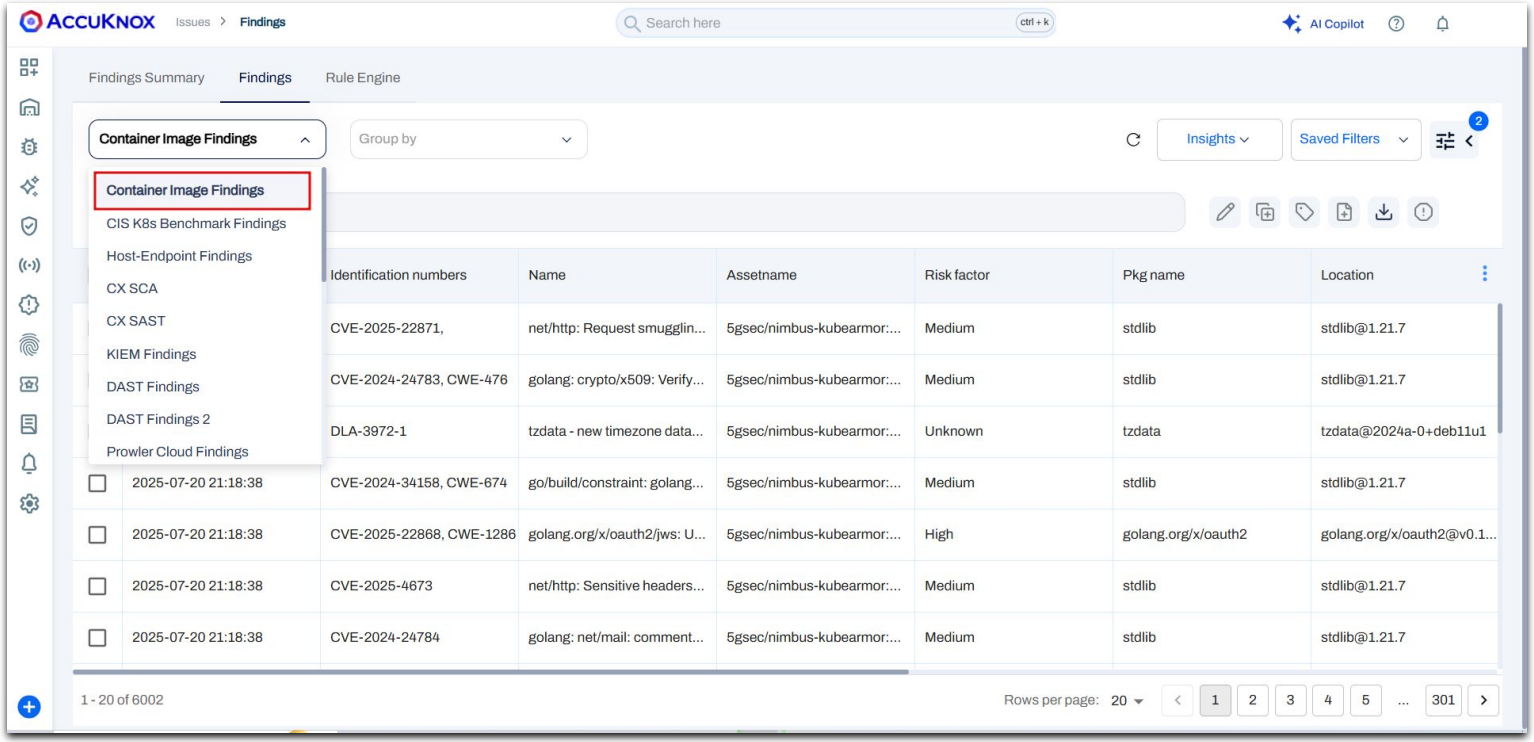
- You can see the findings on the **Registry scan** page.
- Click any of the findings to get more details.





View Findings in Findings page

- Alternatively, you can view the findings on the **Findings** page. Select the **Container Image Findings** to access the relevant details.



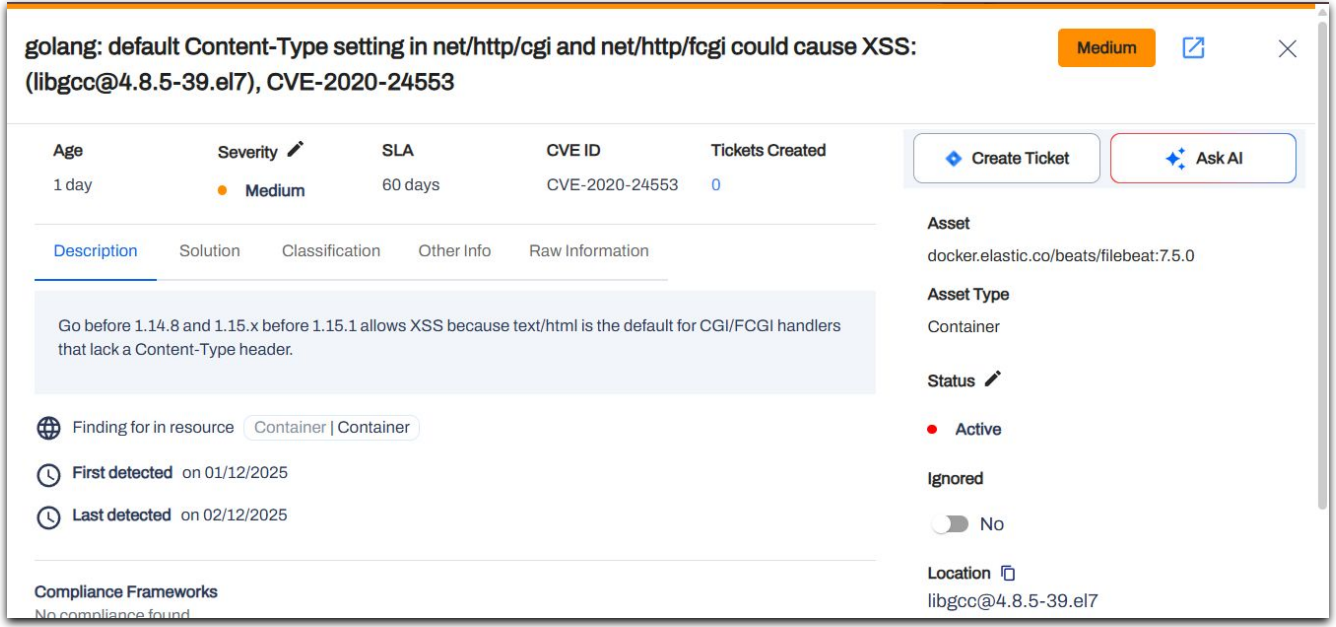
Container Scan Use Cases

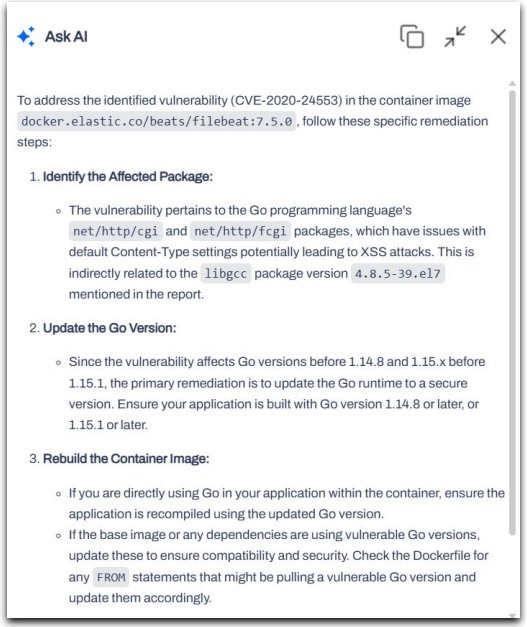
AccuKnox

Use Case 1	**Dependency analysis - Scanning for supply chain vulnerabilities**
Use Case 2	**Scan for sensitive data exposure**
Use Case 3	**Authentication Vulnerabilities**

Use Case 1: Dependency analysis - Scanning for Supply chain vulnerabilities

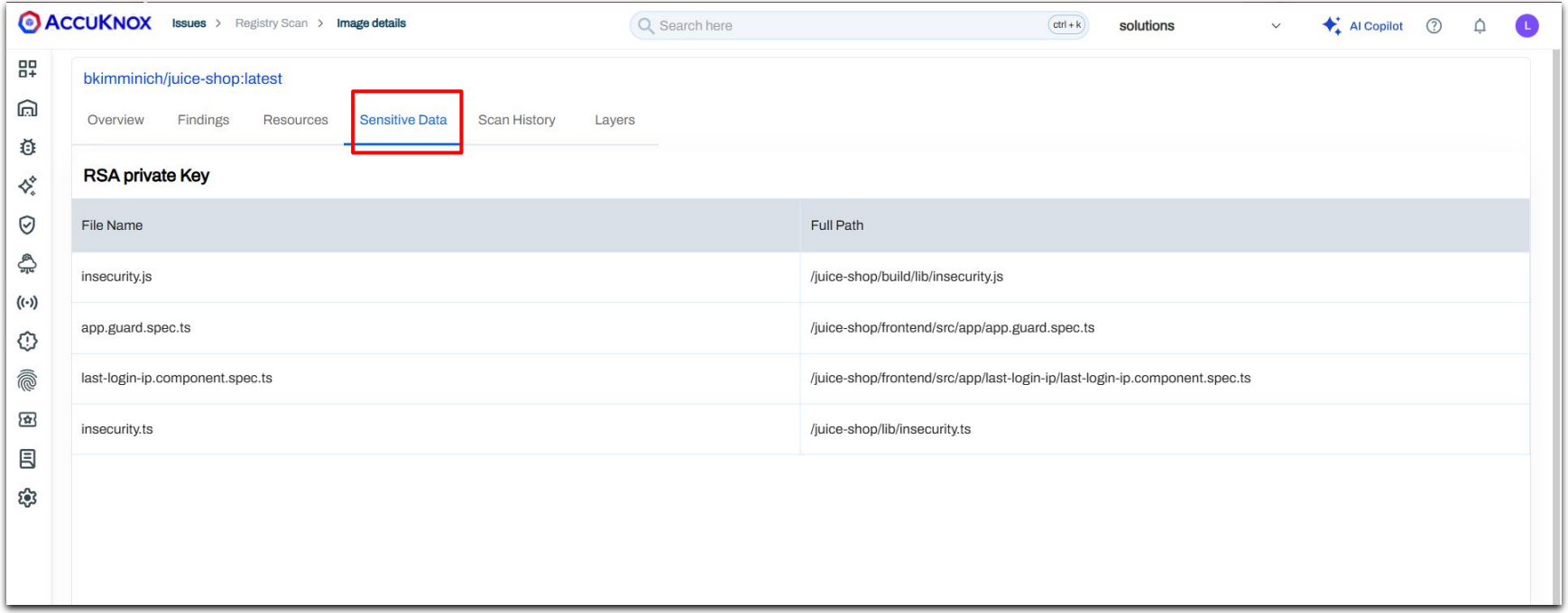
- AccuKnox Container Scan identified a vulnerability in **Golang version 1.14.8.**
- The vulnerability is classified as **Cross-Site Scripting (XSS).**
- AccuKnox provides **actionable remediation guidance and solution** to resolve the issue.





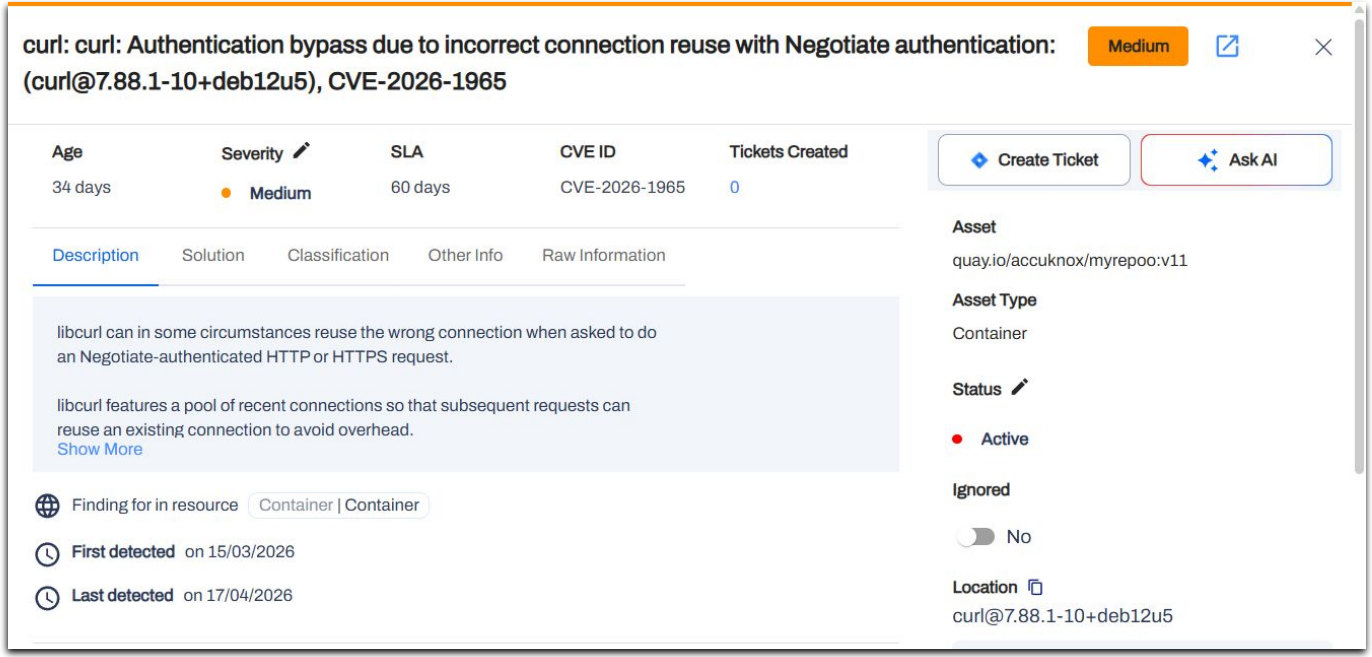
Use Case 2: Scan for Sensitive Data Exposure

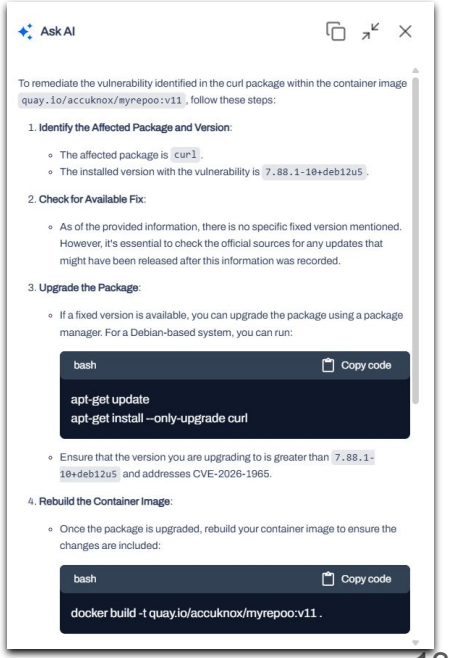
- The container image contains multiple exposed **RSA private keys**
- This represents a critical case of **sensitive data exposure**
- AccuKnox Container Scan detects such sensitive information within images



Use Case 3: Authentication Vulnerabilities

- AccuKnox identified an **authentication bypass vulnerability** in the container image
- The issue is associated with **curl/libcurl (version 7.88.1)**
- This flaw may allow attackers to **bypass authentication mechanisms** under certain conditions
- Proposed solution: Upgrade **curl/libcurl to a patched version** (latest stable release)





AccuKnox

IaC Scan

ASPM Integration

Configuring the IAC scan in GitHub actions

Step 1:Add [AccuKnox IAC scan GitHub action] to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically.github/workflows/your-workflow.yml).
- After the build step, add the AccuKnox IaC scan GitHub Action.

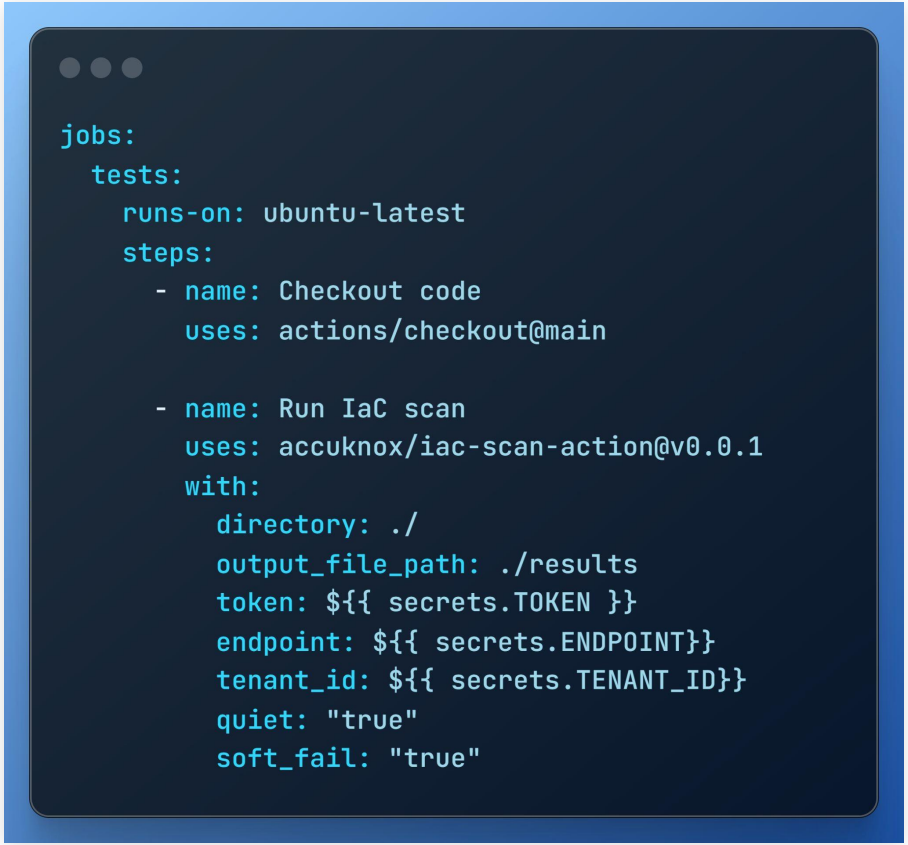
Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select IaC Findings.
- Click on any finding that interests you to view detailed information and recommendations.

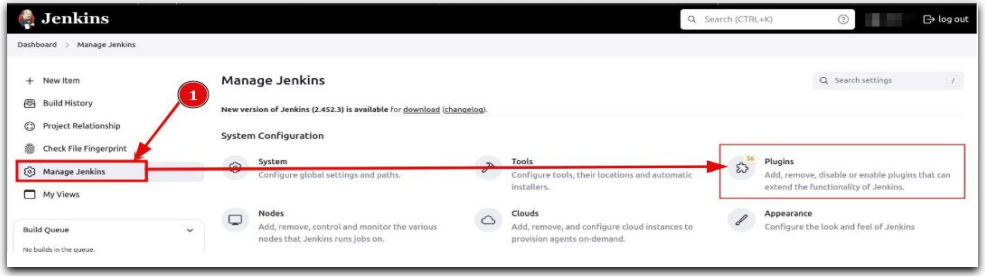
Visit Help Docs for complete guidance

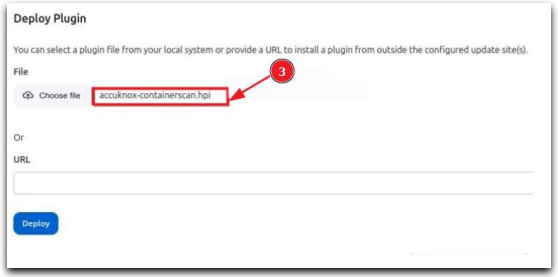


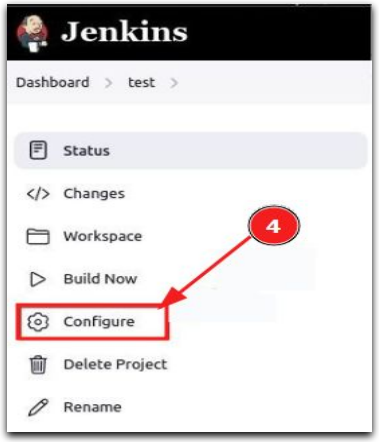
```yaml
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@main

      - name: Run IaC scan
        uses: accuknox/iac-scan-action@v0.0.1
        with:
          directory: ./
          output_file_path: ./results
          token: ${{ secrets.TOKEN }}
          endpoint: ${{ secrets.ENDPOINT}}
          tenant_id: ${{ secrets.TENANT_ID}}
          quiet: "true"
          soft_fail: "true"
```

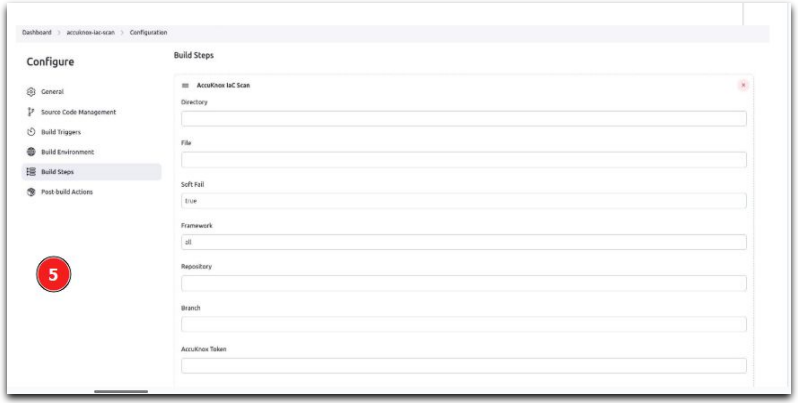
Configuring IaC scan in Jenkins

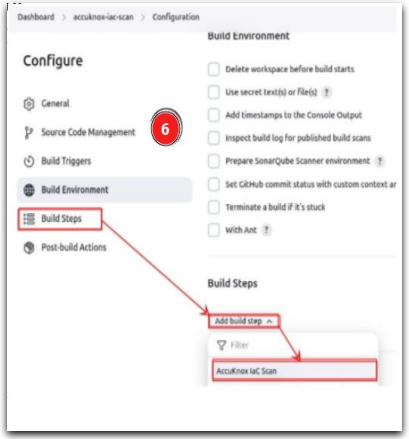
- Jenkins Dashboard → *Manage Jenkins > Plugins > Advanced* → Upload & deploy
- Open your Jenkins job → *Add build step* → Select AccuKnox IaC scan → Fill required details → Trigger the job







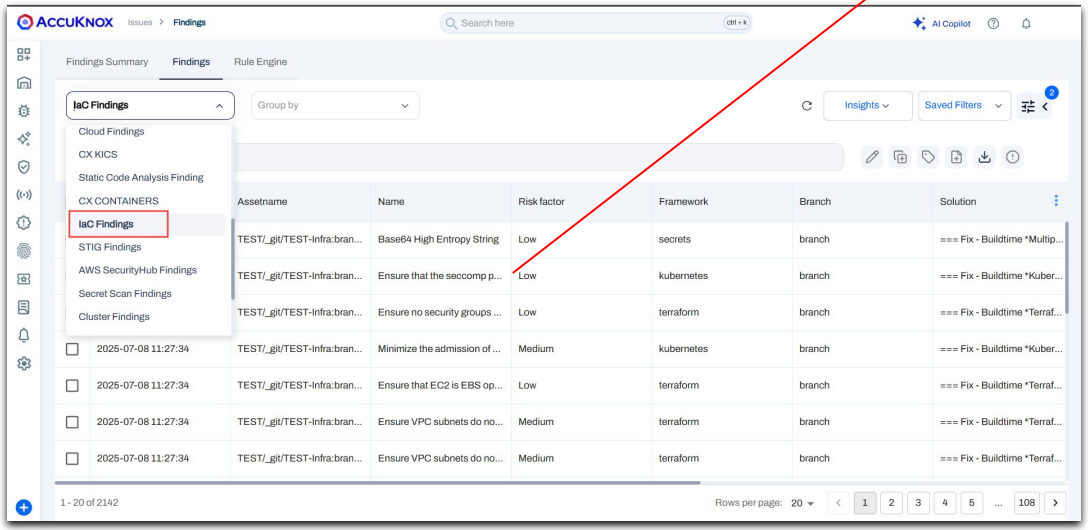


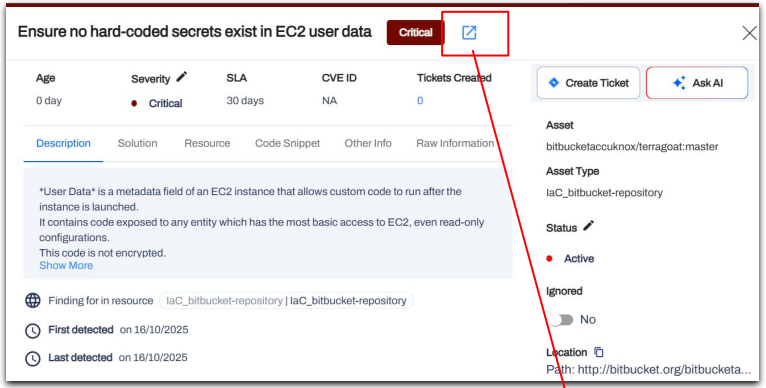


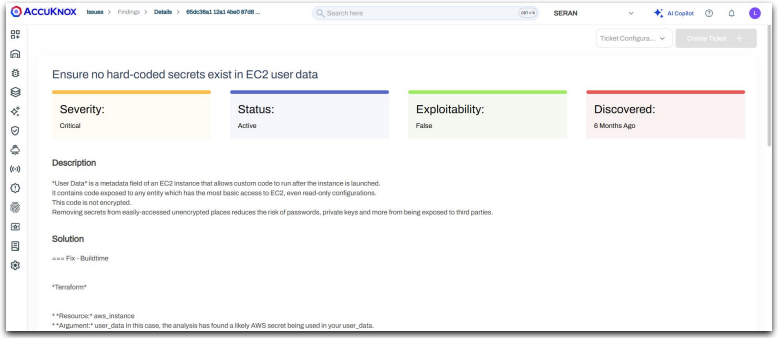
[Visit Help Docs for complete guidance]

View Findings in Findings page

- Go to the AccuKnox > Findings and select the **IAC Findings** from the drop down menu.
- To view detailed information about a finding, click on the finding and then select the arrow icon.







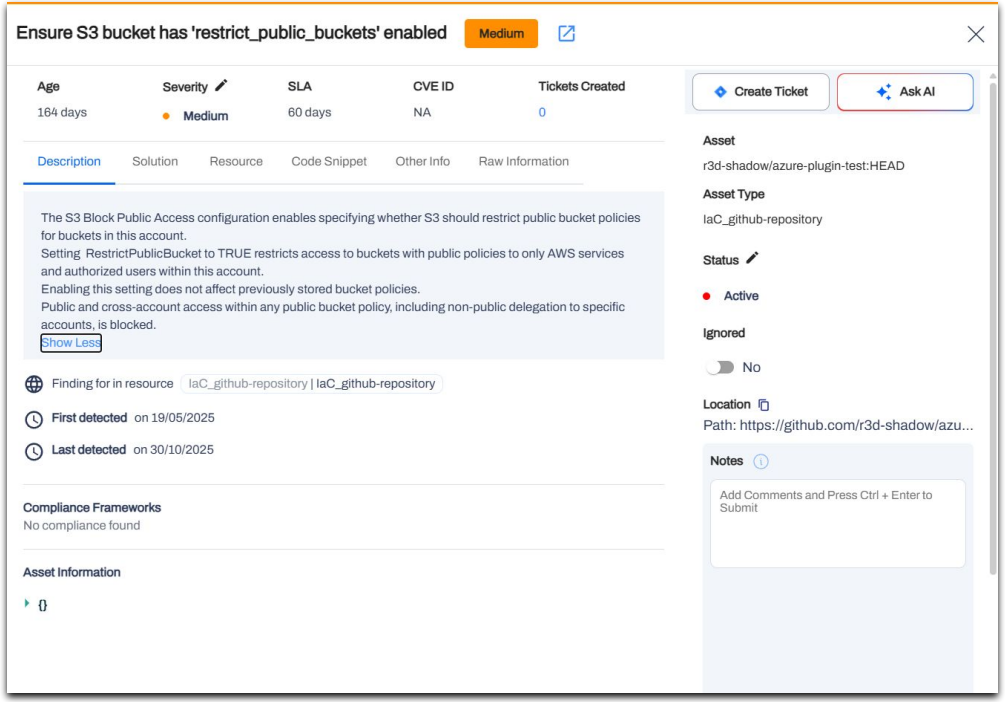
IaC Use Cases

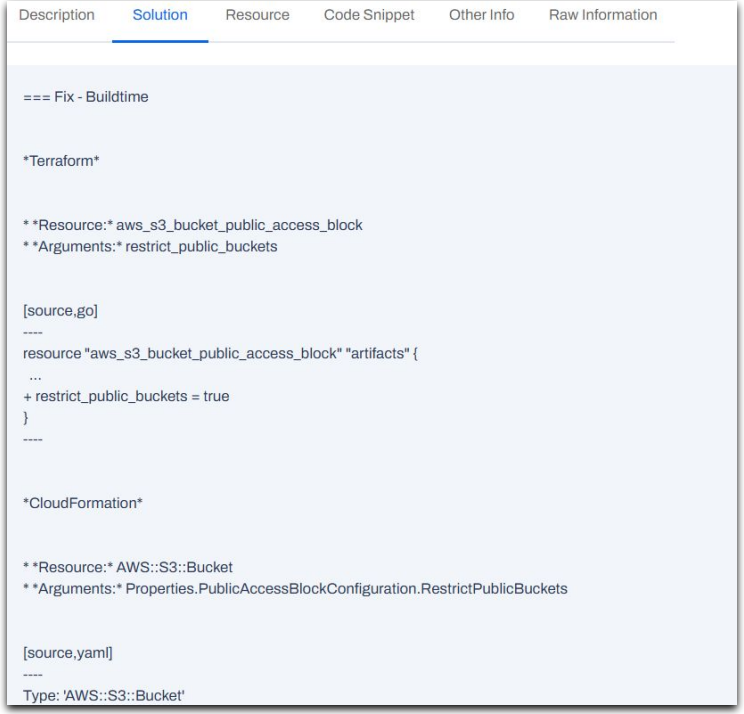
AccuKnox

Use Case 1	Public Cloud Exposure
Use Case 2	Privilege Escalation Enabled
Use Case 3	Credential Exposure

Use case 1: Public Cloud Exposure

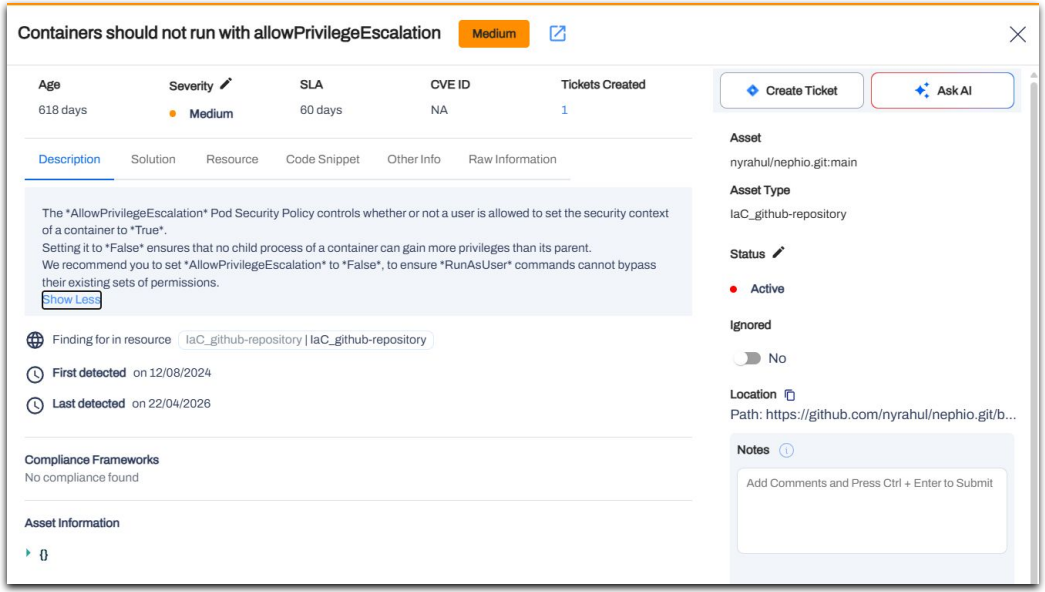
- S3 bucket is misconfigured without restrict_public_buckets enabled, allowing potential public access.
- This can expose sensitive data to the internet, leading to data breaches and security risks.
- AccuKnox IaC scan detects this misconfiguration early in the pipeline and helps fix it before deployment

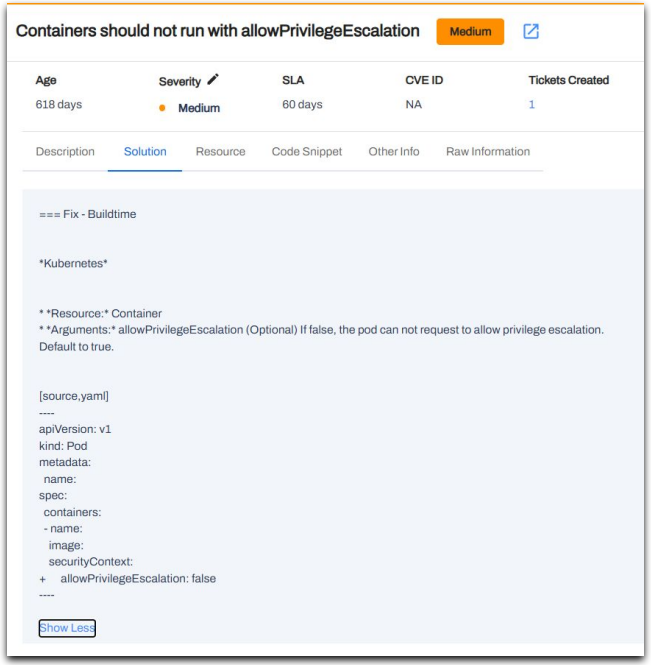




Use case 2: Privilege Escalation Enabled

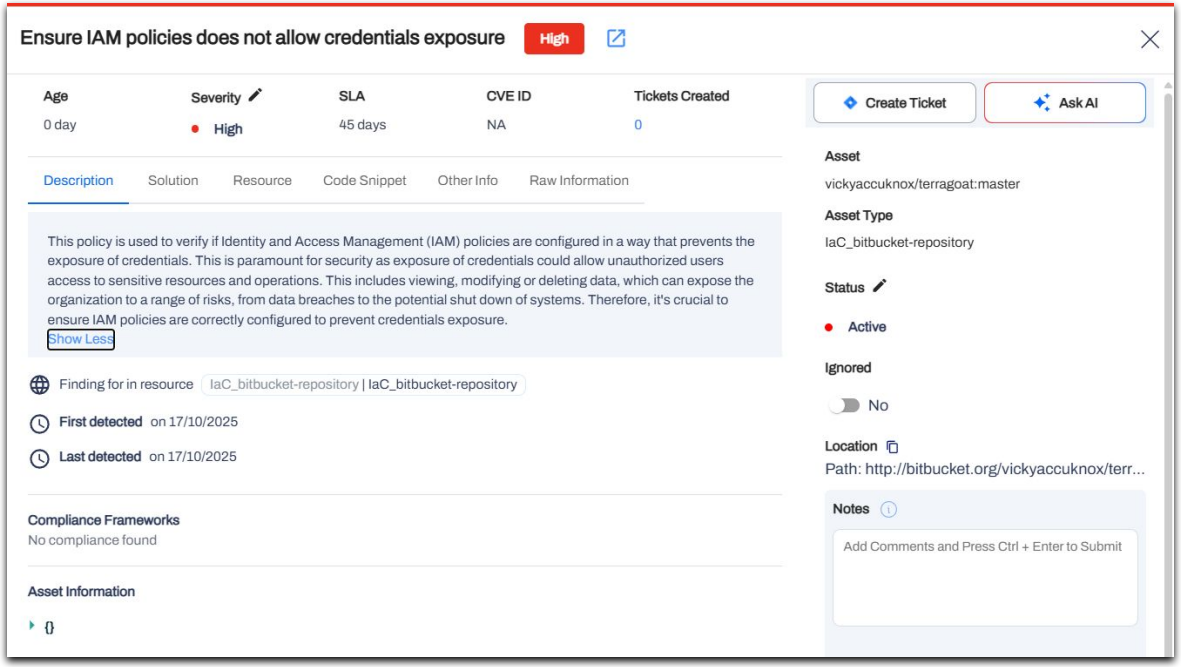
- Containers are configured with allowPrivilegeEscalation enabled, allowing processes to gain higher privileges than intended
- This can lead to privilege escalation attacks, container breakout, and compromise of the underlying host system
- AccuKnox IaC scan detects this insecure setting and enforces disabling privilege escalation to maintain least privilege security

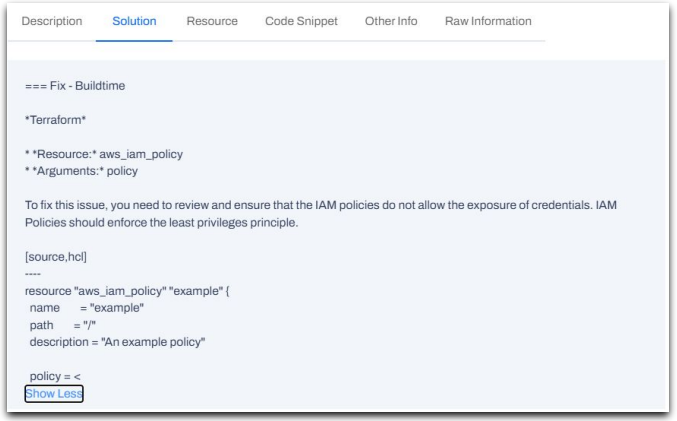




Use case 3: Credential Exposure

- IAM policies are misconfigured in a way that may allow exposure of sensitive credentials
- This can lead to unauthorized access, data breaches, and potential compromise of cloud resources
- AccuKnox IaC scan detects such policy misconfigurations early and helps prevent credential exposure





SAST Scan

ASPM Integration

Configuring the SAST OpenGrep scan in GitHub Actions

ACCUKNOX

Step 1: Add [AccuKnox SAST scan GitHub action] to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically.github/workflows/your-workflow.yml).
- Configure these Parameters as GitHub Secrets `ACCUKNOX_LABEL`,`ACCUKNOX_ENDPOINT`, `ACCUKNOX_TOKEN`

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select SAST Findings.
- Click on any finding that interests you to view detailed information and recommendations.

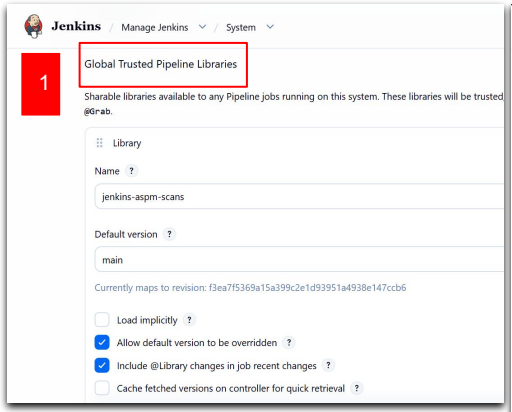
```yaml
name: AccuKnox Opengrep SAST

on:
  push:
    branches:
      - opengrep
  pull_request:
    branches:
      - opengrep

jobs:
  accuknox-cicd:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: "Run AccuKnox SAST: Opengrep"
        uses: accuknox/sast-scan-opengrep-action@v1.0.1
        with:
          accuknox_endpoint: "${{ secrets.ACCUKNOX_ENDPOINT }}"
          accuknox_token: "${{ secrets.ACCUKNOX_TOKEN }}"
          accuknox_label: "${{ secrets.ACCUKNOX_LABEL }}"
          soft_fail: true
```

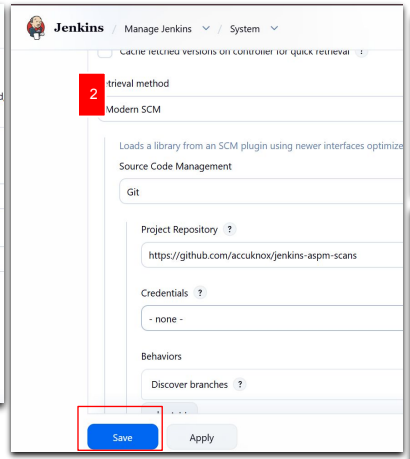
[Visit Help Docs for complete guidance]

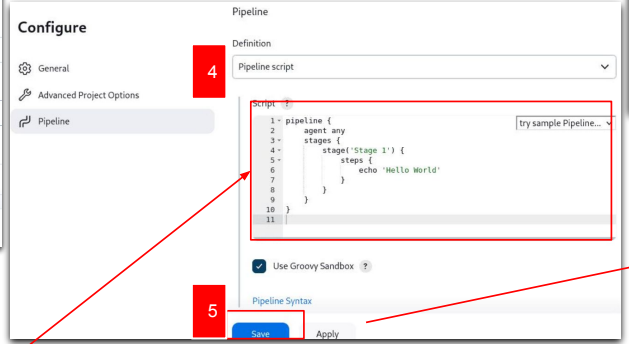
Configuring SAST OpenGrep scan in Jenkins

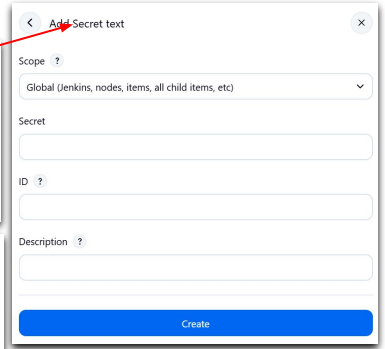
ACCUKNOX

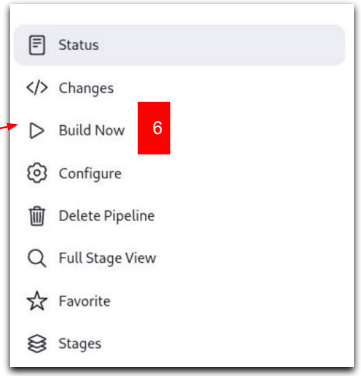
- Manage Jenkins → System → Global Trusted Pipeline Libraries
- Manage Jenkins → Credentials → Global → Add Credentials
- Go to Configure → pipeline → In the script box that appears, paste your pipeline script.
- After adding the script, click "Save" or "Apply."
- Go to the newly created pipeline and click "Build Now" to run the script.









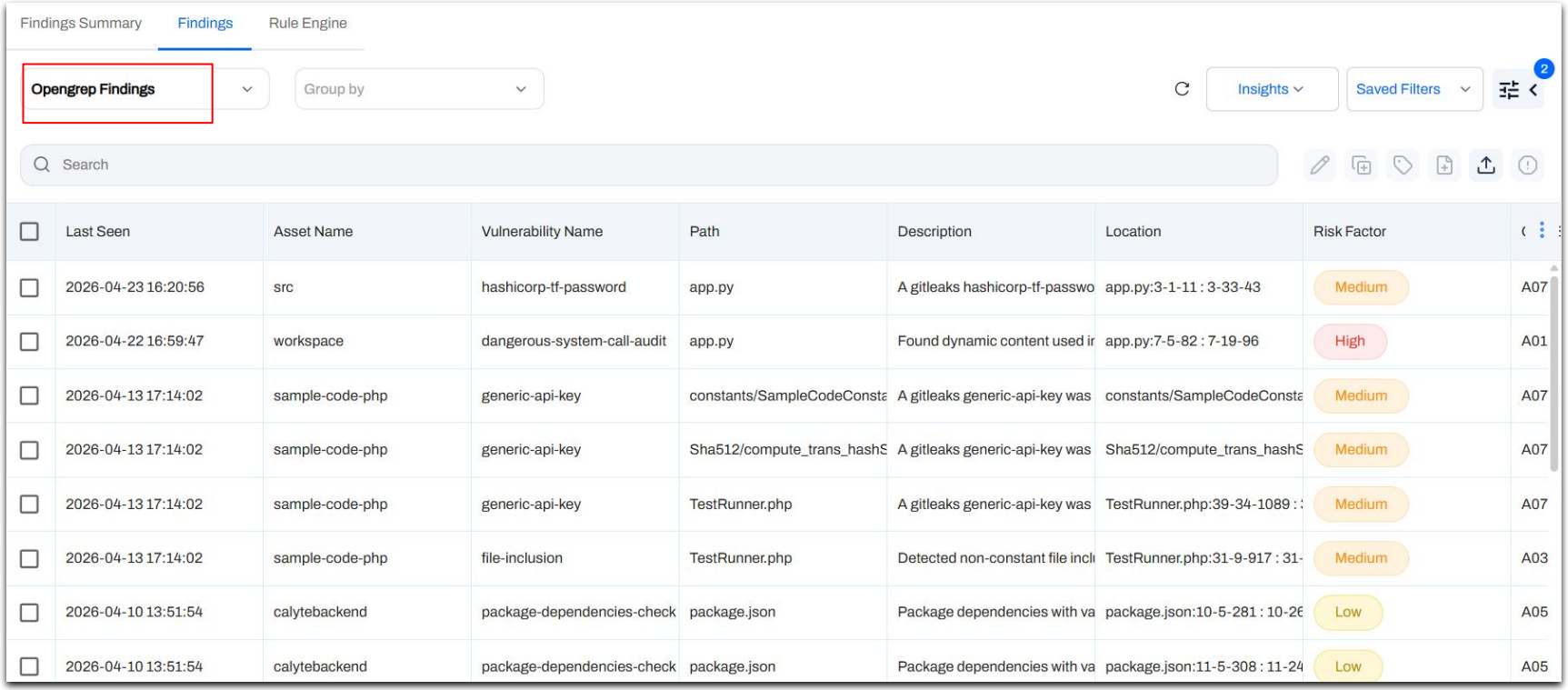



```
@Library('jenkins-aspm-scans@main') _

pipeline {
    agent any
    environment {
        ACCUKNOX_ENDPOINT = credentials('accuknox-endpoint')
        ACCUKNOX_LABEL    = credentials('accuknox-label')
        ACCUKNOX_TOKEN    = credentials('accuknox-token')
    }
    stages {
        stage('Checkout') {
            steps {
```

[Visit Help Docs for complete guidance]

View Findings in Findings page

- Go to the AccuKnox > Findings and select the **OpenGrep findings** here.



Last Seen	Asset Name	Vulnerability Name	Path	Description	Location	Risk Factor	
2026-04-23 16:20:56	src	hashicorp-tf-password	app.py	A gitleaks hashicorp-tf-passwo	app.py:3-1-11 : 3-33-43	Medium	A07
2026-04-22 16:59:47	workspace	dangerous-system-call-audit	app.py	Found dynamic content used ir	app.py:7-5-82 : 7-19-96	High	A01
2026-04-13 17:14:02	sample-code-php	generic-api-key	constants/SampleCodeConsta	A gitleaks generic-api-key was	constants/SampleCodeConsta	Medium	A07
2026-04-13 17:14:02	sample-code-php	generic-api-key	Sha512/compute_trans_hashS	A gitleaks generic-api-key was	Sha512/compute_trans_hashS	Medium	A07
2026-04-13 17:14:02	sample-code-php	generic-api-key	TestRunner.php	A gitleaks generic-api-key was	TestRunner.php:39-34-1089 :	Medium	A07
2026-04-13 17:14:02	sample-code-php	file-inclusion	TestRunner.php	Detected non-constant file incl	TestRunner.php:31-9-917 : 31-	Medium	A03
2026-04-10 13:51:54	calytebackend	package-dependencies-check	package.json	Package dependencies with va	package.json:10-5-281 : 10-26	Low	A05
2026-04-10 13:51:54	calytebackend	package-dependencies-check	package.json	Package dependencies with va	package.json:11-5-308 : 11-24	Low	A05

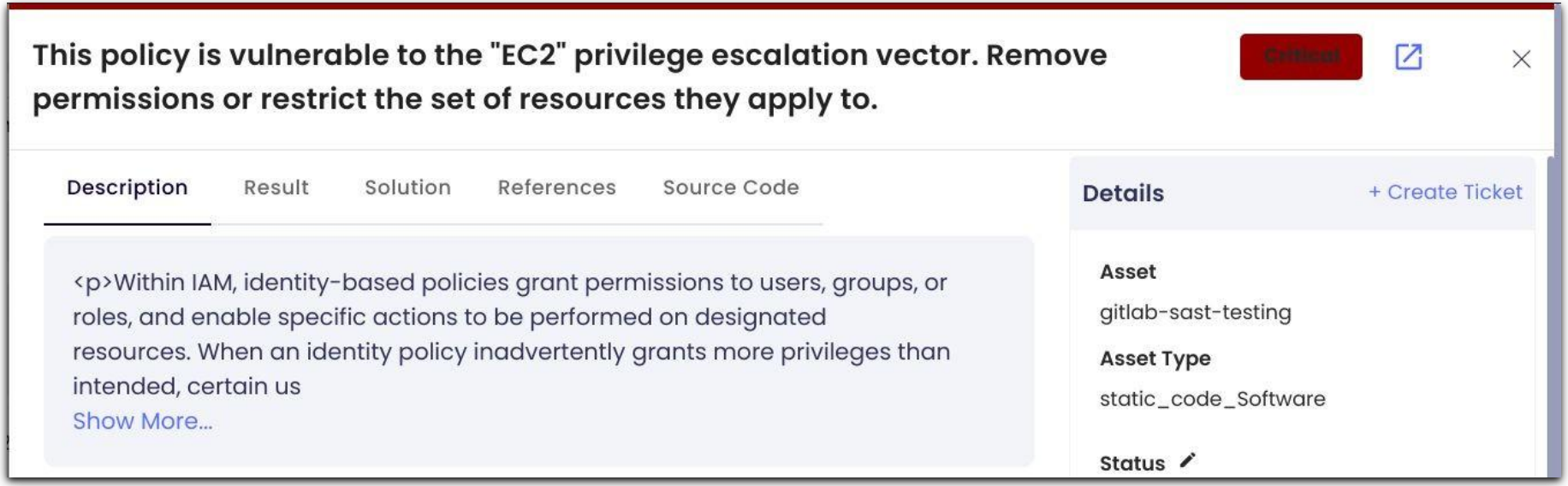
SAST Use Cases

AccuKnox

Use Case 1	Privilege Escalation
Use Case 2	XML External Entity(XXE) Injection
Use Case 3	Hard Coded Password

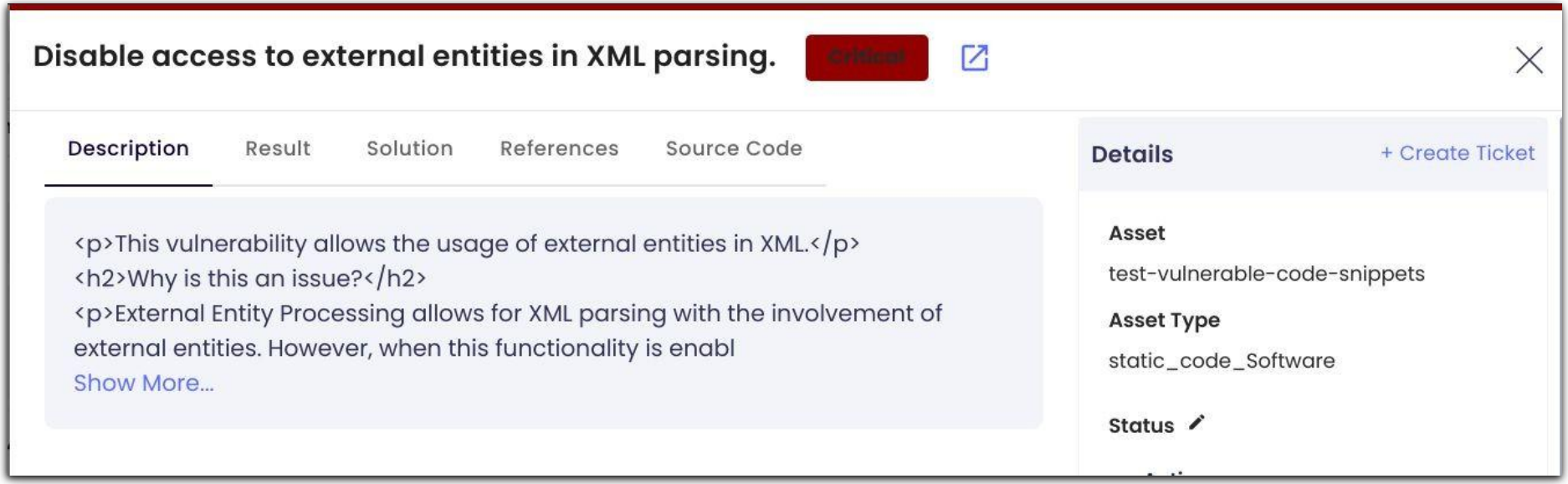
Use case 1: Privilege Escalation

- An IAM policy in this code is vulnerable to privilege escalation attack.
- AccuKnox identifies this vulnerability and suggests a solution.



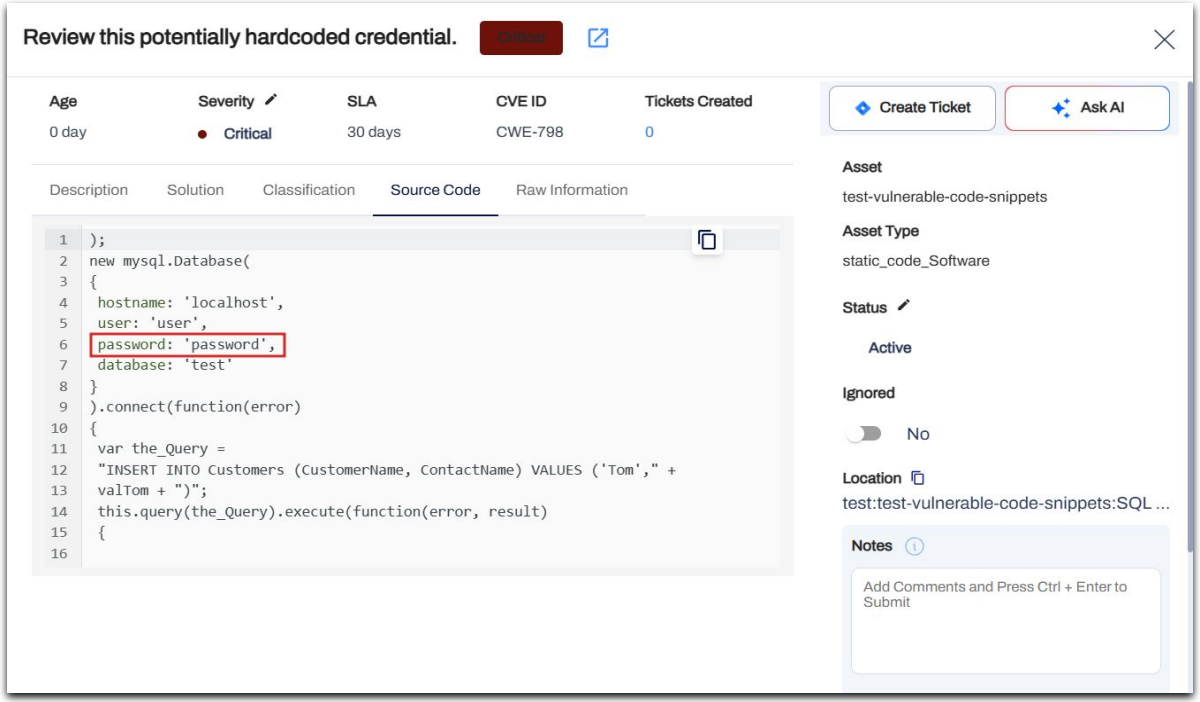
Use case 2: XML External Entity (XXE) injection

- This code have an XML parsing vulnerability XML External Entity injection.
- AccuKnox identifies the vulnerability and proposes a solution.



Use case 3: Hard Coded Password

- There is a hardcoded password in the source code.
- AccuKnox identifies the vulnerability, and proposes a solution.



AccuKnox

DAST Scan

ASPM Integration

Configuring the DAST scan in GitHub actions

ACCUKNOX

Step 1: Add [AccuKnox DAST scan GitHub action] to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically.github/workflows/your-workflow.yml)
- After the build step, add the AccuKnox container scan GitHub Action.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox:

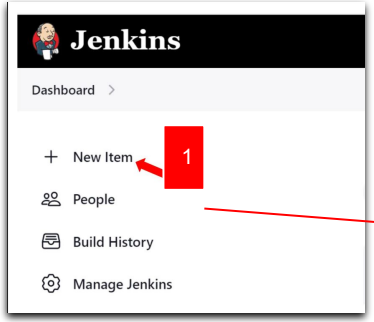
- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select DAST Findings.
- Click on any finding that interests you to view detailed information and recommendations.

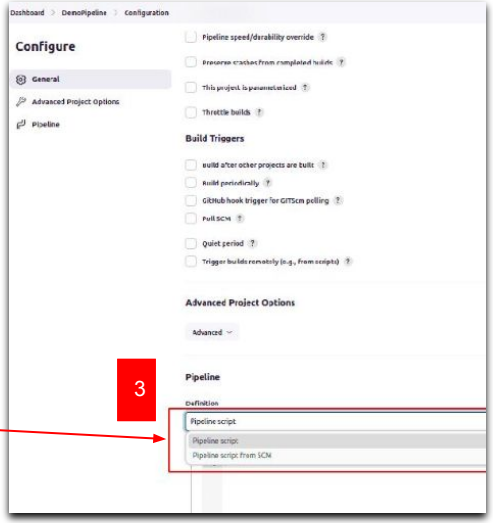
Visit Help Docs for complete guidance

```
jobs:
  zap:
    runs-on: ubuntu-latest
    steps:
      - name: set permissions
        run: sudo chmod -R 777 .

      - name: workspace permissions
        run: sudo chmod -R 777 ${{ github.workspace }}

      - name: zap scan
        run: |
          docker run --rm -v ${{ github.workspace }}:/zap/wrk/:rw -t zaproxy/zap-stable zap-
baseline.py \
            -t <url that you want to scan> \
            -J report.json
            -I

      - name: zap
        run: |
          cd ${GITHUB_WORKSPACE}
          curl --location --request POST 'https://${{ secrets.accuknox_url
}}/api/v1/artifact/?tenant_id=${{ secrets.tenant_id
}}&data_type=ZAP&label_id=dasttest&save_to_s3=false' \
            --header 'Tenant-Id: ${{ secrets.tenant_id }}' \
            --header 'Authorization: Bearer ${{ secrets.token }}'' \
            --form 'file=@\"report.json"'
```

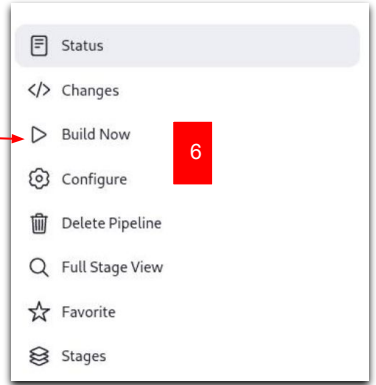
Configuring the DAST scan in Jenkins

- Go to Jenkins Dashboard → Click *New Item* → Enter name → Select *Pipeline* → Click OK
- In Pipeline configuration → Select *Pipeline script* → Paste your script → Save/Apply
- Open the pipeline → Click *Build Now* to execute





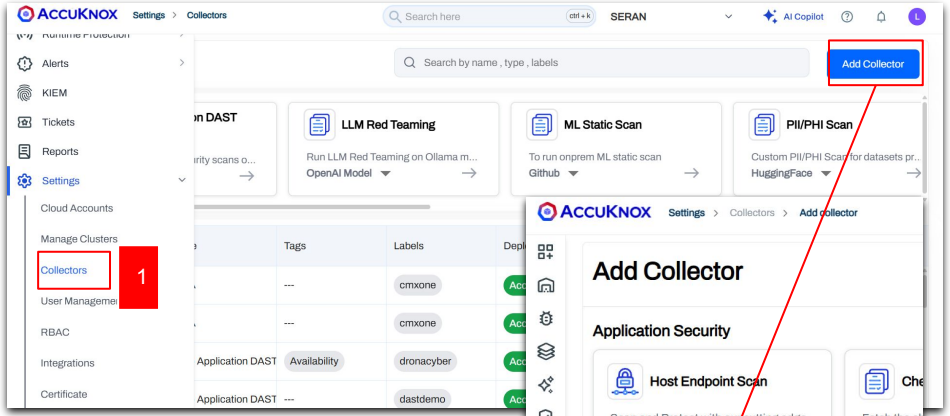

```
pipeline {
  agent any

  environment {
    TOKEN     = credentials('ACCUKNOX_TOKEN')
    END_POINT = 'cspm.demo.accuknox.com'
    TENANT_ID = '167'
    LABEL     = 'SPOC'
  }

  stages {
    stage('Checkout Repository') {
      steps {
        git 'https://github.com/safeer-accuknox/mfa-dast-
      }
```

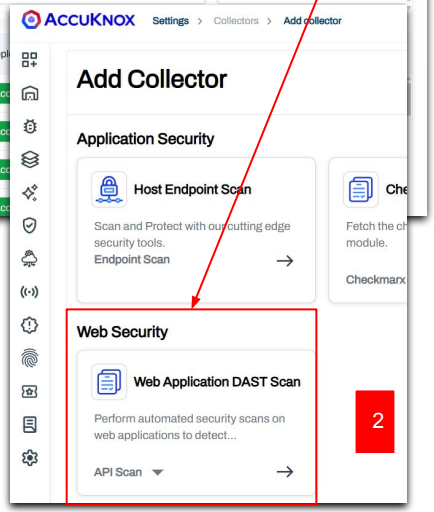


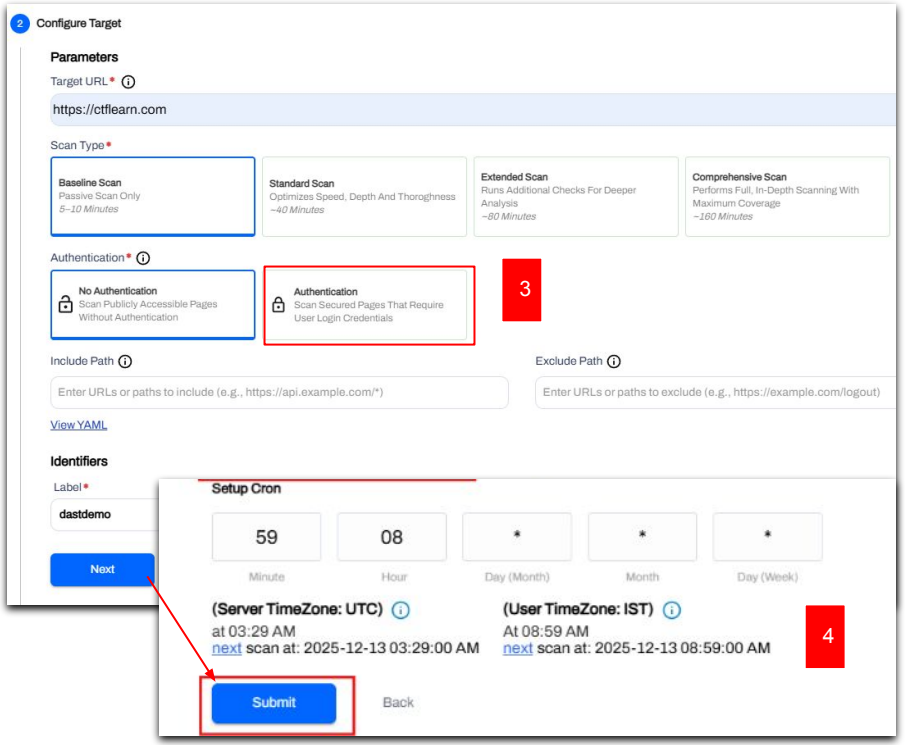
[Visit Help Docs for complete guidance]

Authenticated/Non Authenticated DAST Collector Scan

- Go to *AccuKnox → Settings > Collectors > Add Collector →* Select **Web Application DAST Scan** and give a name
- Select **Authenticated** (enter Login URL & credentials) or **Non-Authenticated** scan
- Fill required scan settings and target details.
- Set cron job (if needed) and click **Submit**





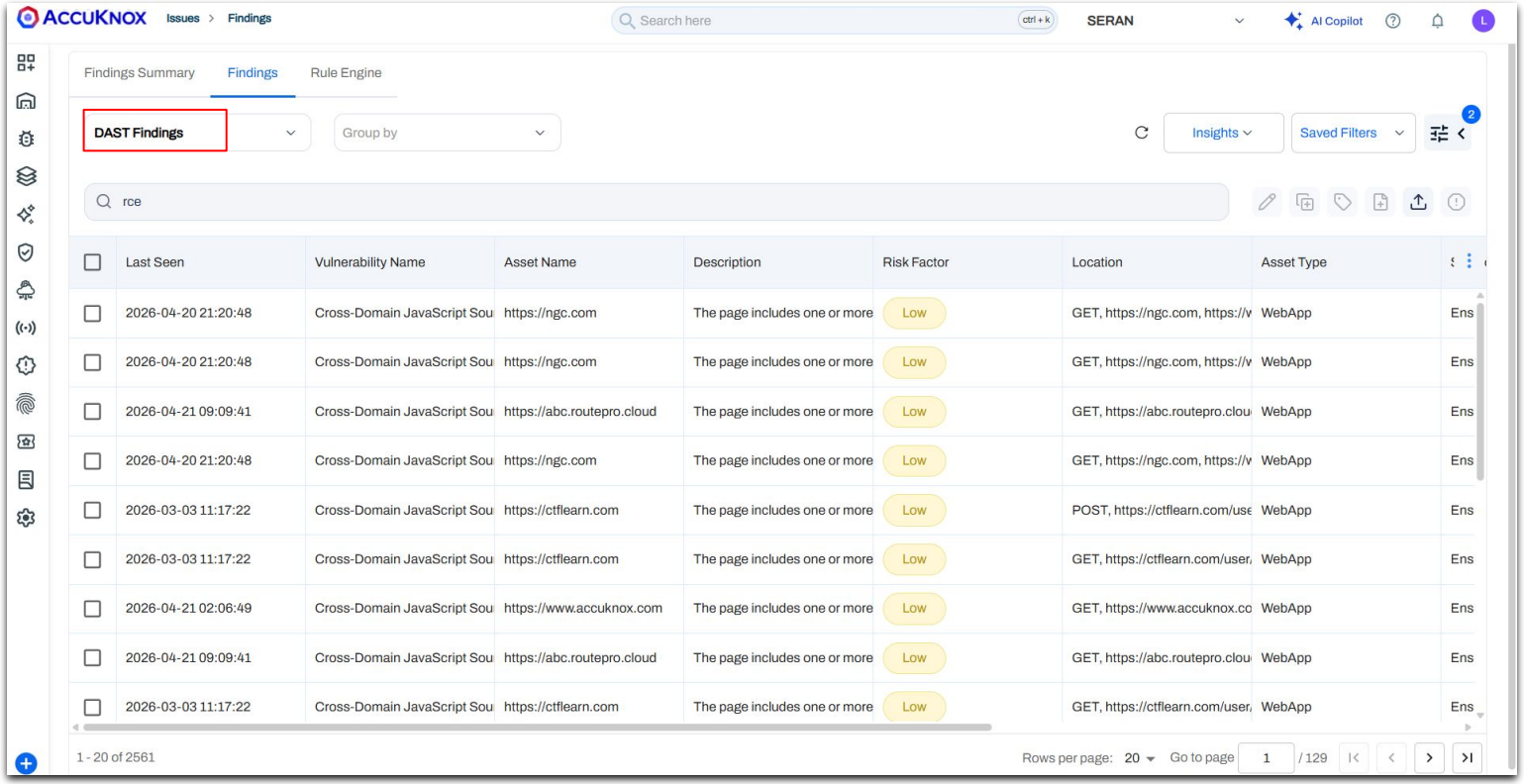


Visit help Docs for Complete Guidance:

- [Authenticated Scan]
- [Non-authenticated Scan]

View Findings in Findings page

- To get the findings, go to the AccuKnox > Findings and select the **DAST findings here.**

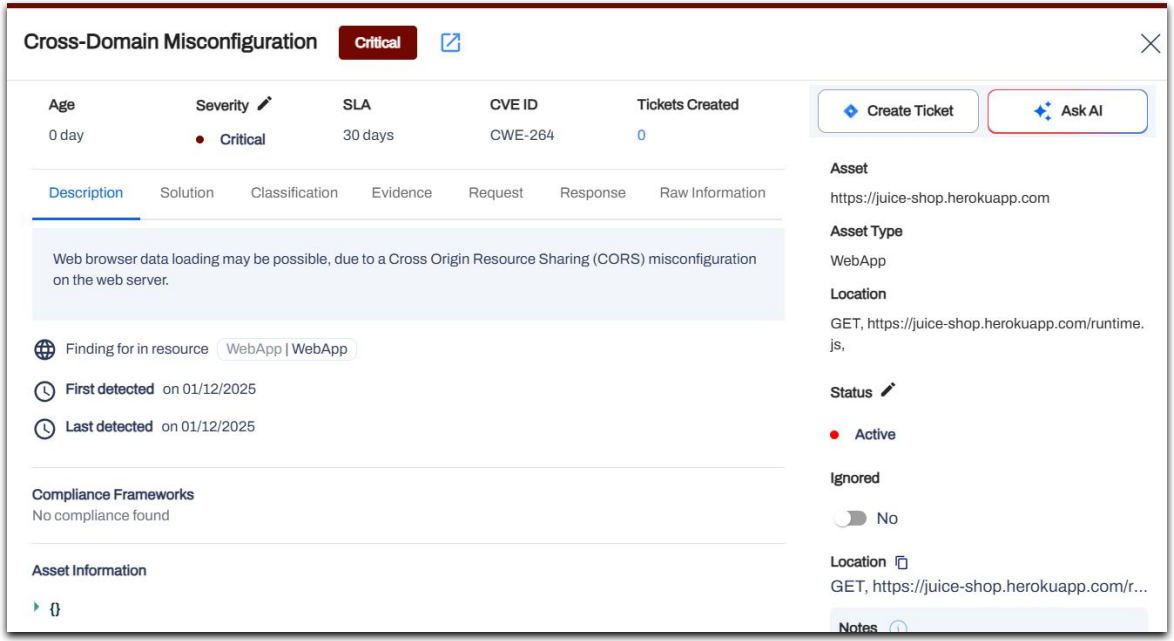


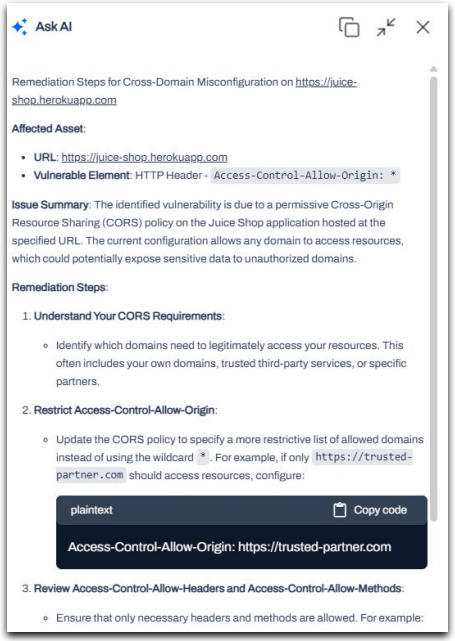
DAST Use Cases

Use Case 1	Cross Origin Resource Sharing (CORS) Misconfiguration
Use Case 2	File Inclusion Vulnerability
Use Case 3	Cross site scripting vulnerability

Use case 1: Cross Origin Resource Sharing (CORS) misconfiguration

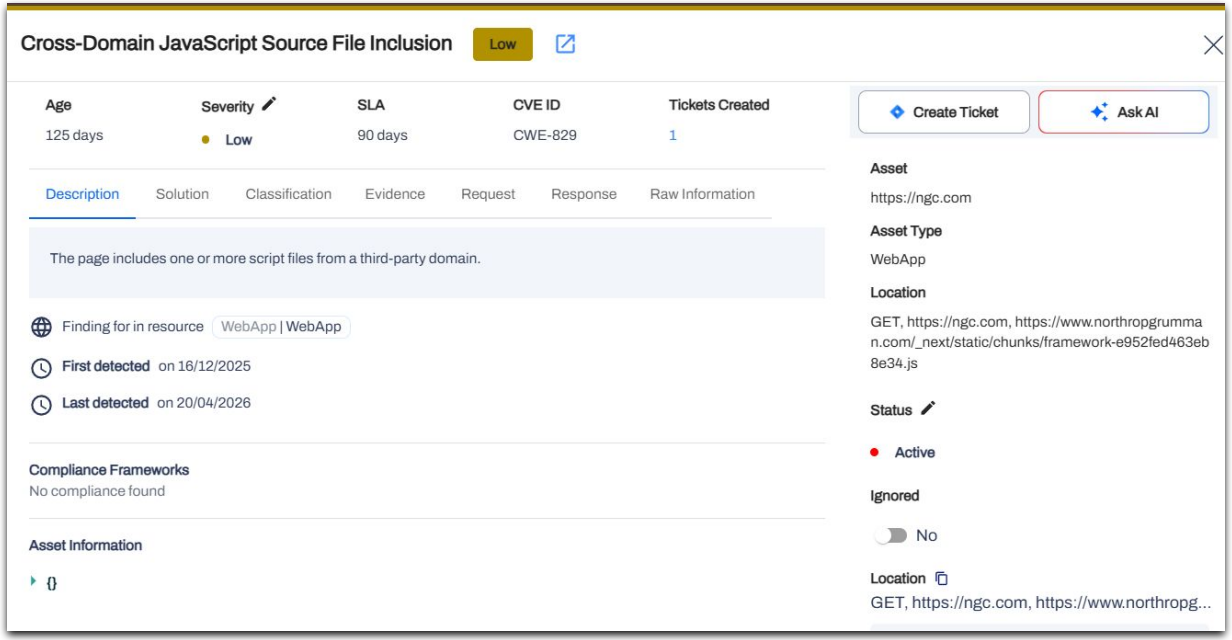
- This web application is vulnerable to CORS.
- CORS can lead to so many issues such as unauthorized access, cross site scripting, session hijacking and many other issues.
- AccuKnox identifies this vulnerability and proposes the solution.

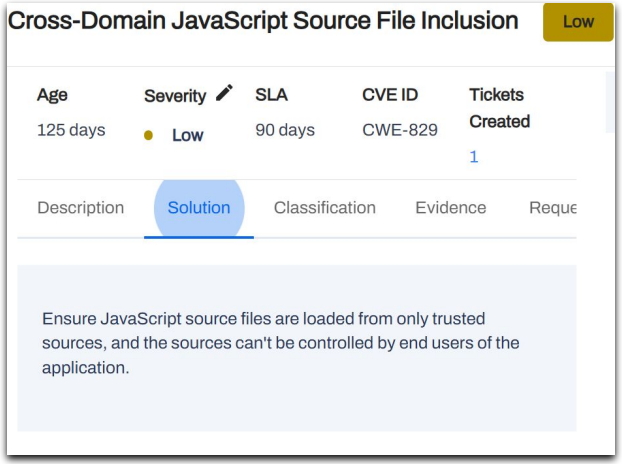




Use case 2: File inclusion vulnerability

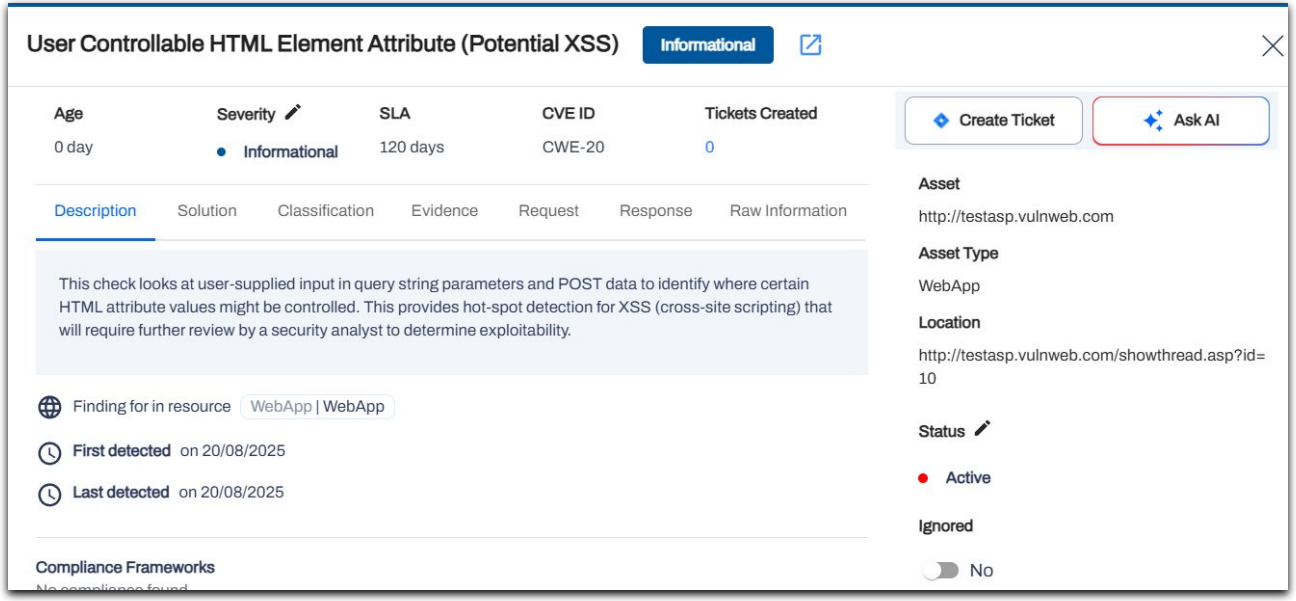
- This web application is vulnerable to file inclusion.
- In file inclusion attack, and attacker can access files stored on a web server.
- This can lead to sensitive data leakage and source code leakage.
- AccuKnox identifies this vulnerability, and proposes a solution.

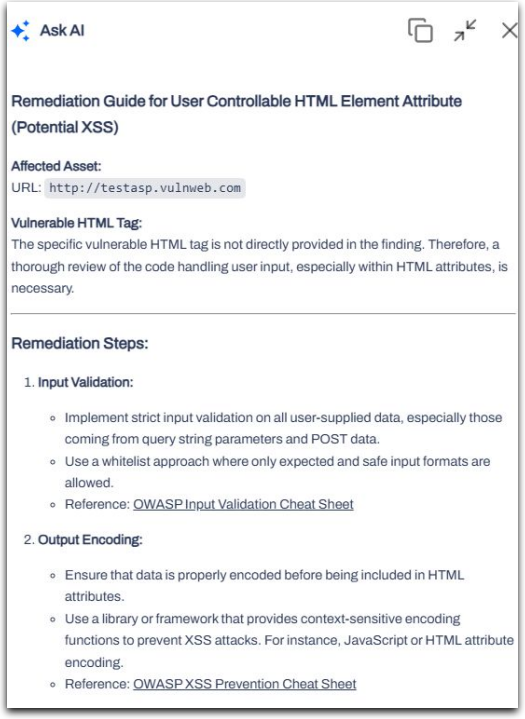




Use case 3: Cross site scripting vulnerability

- This web application have a cross site scripting vulnerability (XSS)
- XSS allows an attacker to inject arbitrary javascript code into a webpage
- AccuKnox identifies the vulnerability and proposes a solution to it





Secret Scanning

ASPM Integration

Configuring the Secret scan in GitHub actions

Step 1:Add [AccuKnox Secret Scan GitHub action] to your workflow like this image.

- Open your GitHub repository and navigate to your workflow file (typically.github/workflows/your-workflow.yml).
- After the build step, add the AccuKnox container scan GitHub Action.

Step 2: Run the Workflow

- Push your changes to trigger the workflow, or manually run it from the "Actions" tab in your repository.

Step 3: Review Findings in AccuKnox

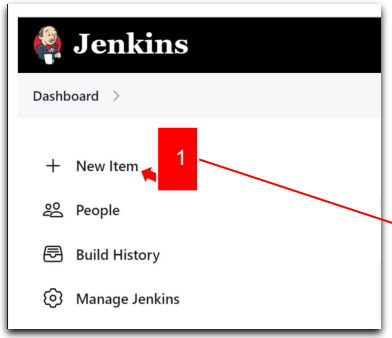
- Log in to your AccuKnox dashboard and navigate to the Issues section.
- Go to the "Findings" tab and select Secret scan Findings.
- Click on any finding that interests you to view detailed information and recommendations.

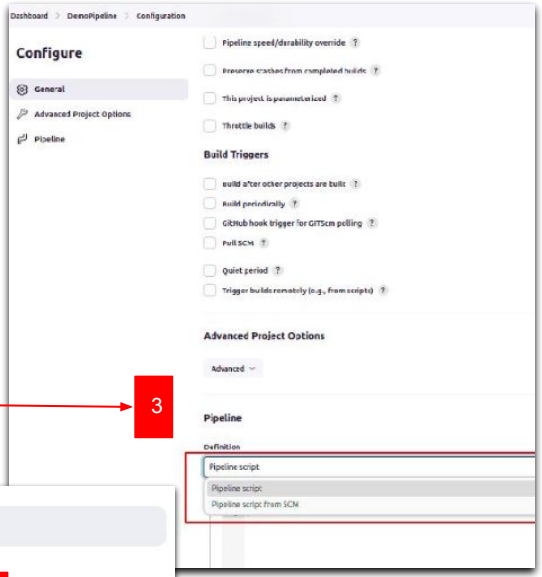
Visit Help Docs for complete guidance

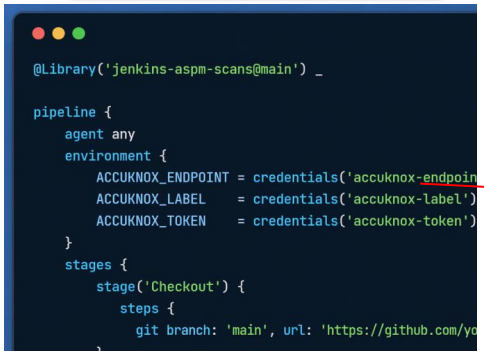
```yaml
name: AccuKnox Secret Scan Workflow
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: Run Secret Scan
        uses: accuknox/secret-scan-action@latest
        with:
          branch: "main"
          results: ""
          exclude_paths: "tests/,docs/"
```

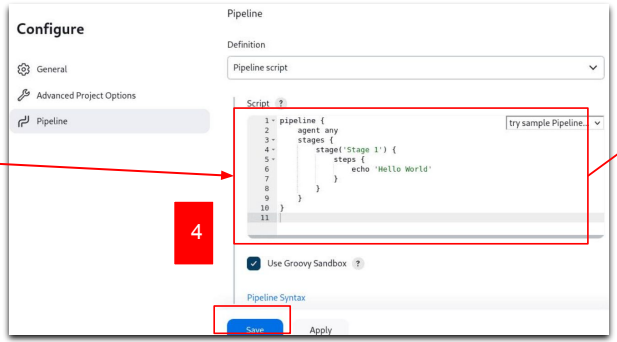
Configuring the Secret Scan in Jenkins

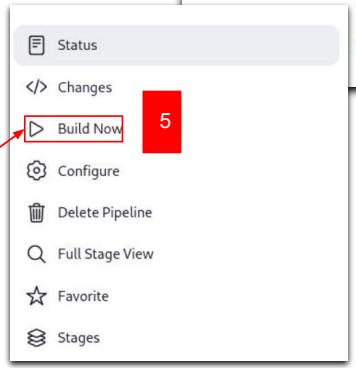
- Go to Jenkins Dashboard → Click *New Item* → Enter name → Select *Pipeline* → Click OK
- In Pipeline configuration → Select *Pipeline script* → Paste your script → Save/Apply
- Open the pipeline → Click *Build Now* to execute







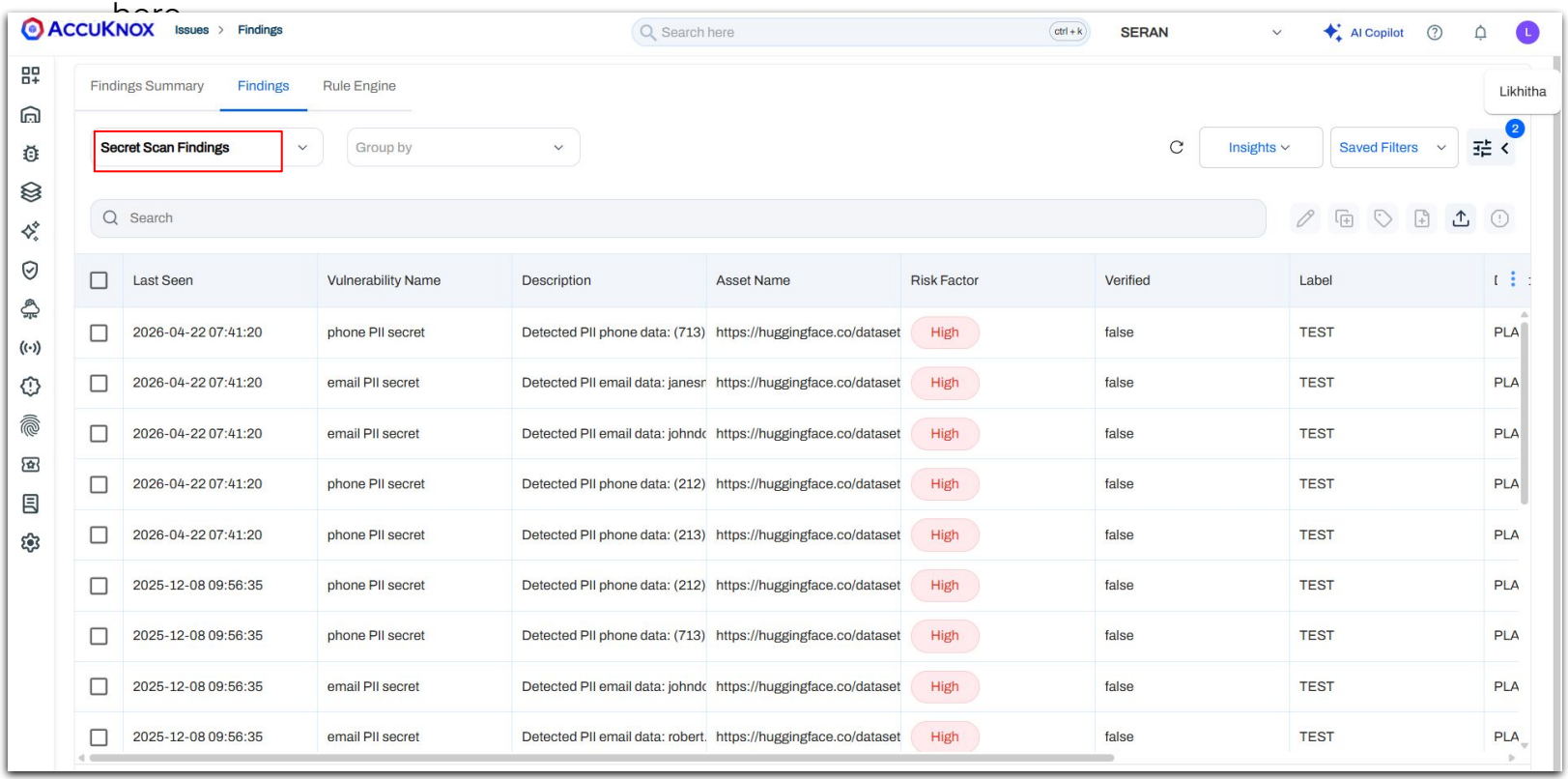




[Visit Help Docs for complete guidance]

View Findings in Findings page

- To get the findings, go to the AccuKnox > Findings and select **Secret Scan Findings** here.



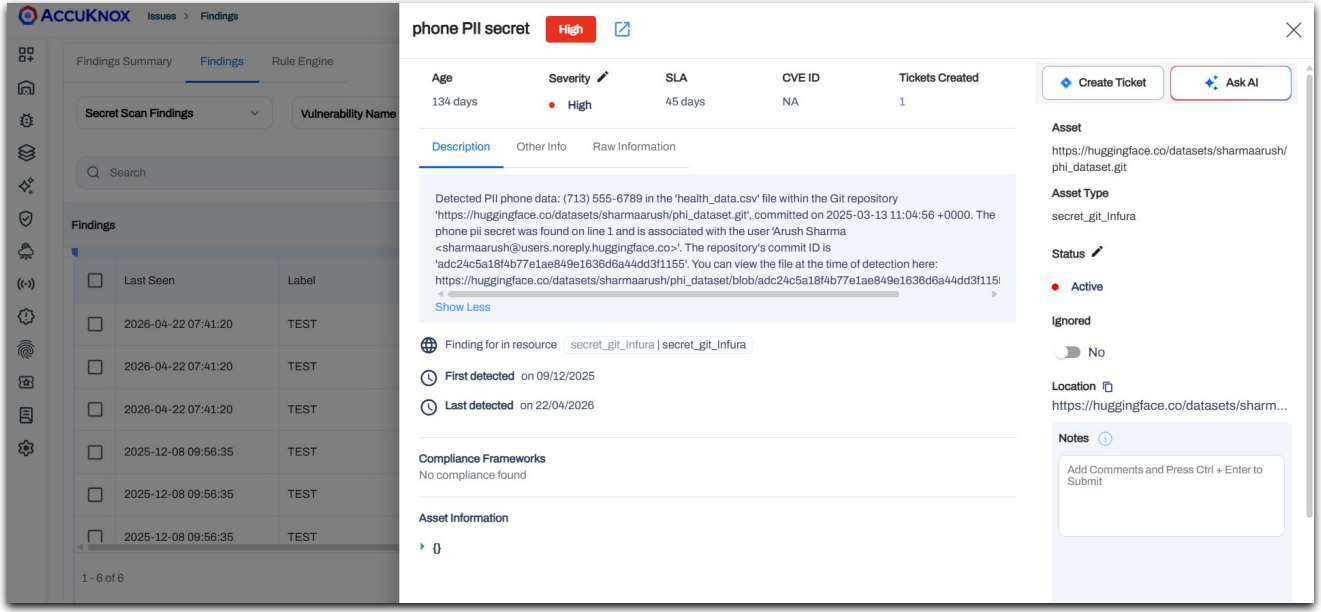
Secret Scan Use Cases

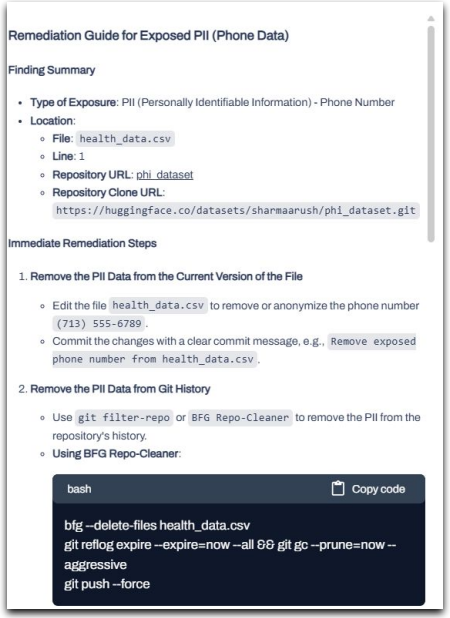
AccuKnox

Use Case 1	Sensitive Data Exposure (PII Leakage)
Use Case 2	API Keys & Access Tokens Leakage
Use Case 3	Private Key Exposure

Use case 1: Sensitive Data Exposure (PII Leakage)

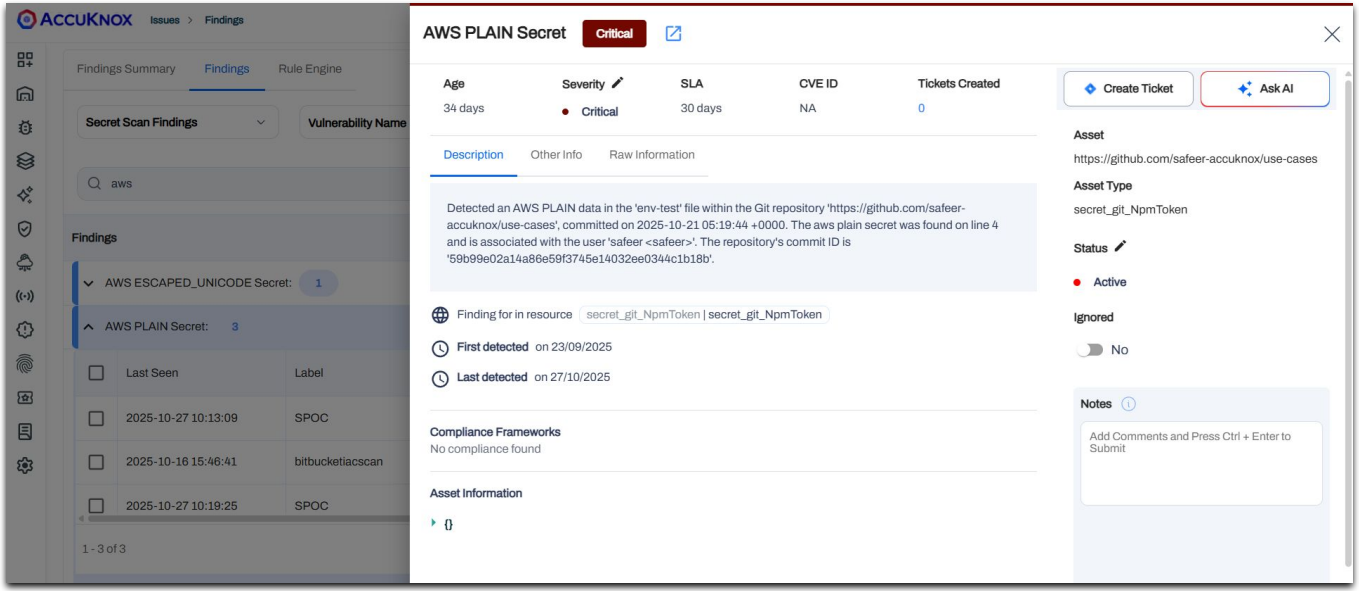
- AccuKnox identified exposed PII data (phone number) in the repository
- This may lead to privacy violations, data misuse, and compliance risks (GDPR, etc.)
- AccuKnox identifies the vulnerability and proposes a solution to it

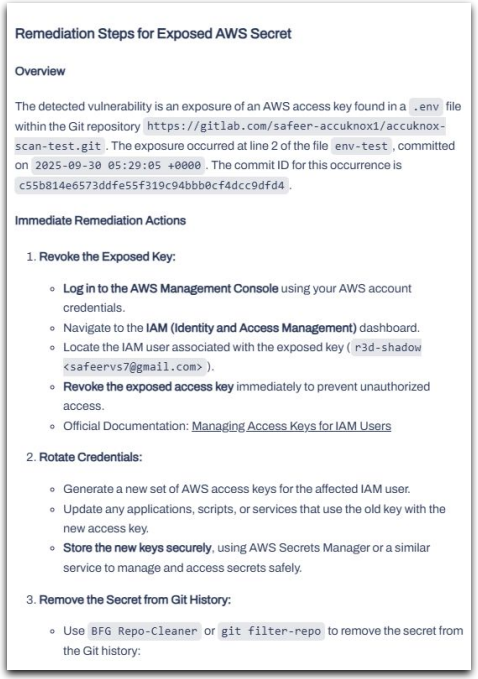




Use case 2: API Keys & Access Tokens Leakage

- AccuKnox detected exposed AWS secret key in the repository
- This may lead to unauthorized cloud access, resource misuse, and financial loss
- AccuKnox proposes the solution to remove the key, revoke and rotate credentials, and use secure secret management.





Remediation Steps for Exposed AWS Secret

Overview

The detected vulnerability is an exposure of an AWS access key found in a `.env` file within the Git repository `https://gitlab.com/safeer-accuknox1/accuknox-scan-test.git`. The exposure occurred at line 2 of the file `env-test`, committed on `2025-09-30 05:29:05 +0000`. The commit ID for this occurrence is `c55b814e6573ddfe55f319c94bbb0cf4dcc9dfd4`.

Immediate Remediation Actions

1. **Revoke the Exposed Key:**
   - **Log in to the AWS Management Console** using your AWS account credentials.
   - Navigate to the **IAM (Identity and Access Management)** dashboard.
   - Locate the IAM user associated with the exposed key (`r3d-shadow <safeervs7@gmail.com>`).
   - **Revoke the exposed access key** immediately to prevent unauthorized access.
   - Official Documentation: Managing Access Keys for IAM Users
2. **Rotate Credentials:**
   - Generate a new set of AWS access keys for the affected IAM user.
   - Update any applications, scripts, or services that use the old key with the new access key.
   - **Store the new keys securely**, using AWS Secrets Manager or a similar service to manage and access secrets safely.
3. **Remove the Secret from Git History:**
   - Use `BFG Repo-Cleaner` or `git filter-repo` to remove the secret from the Git history:

Use case 3: Private Key Exposure

- AccuKnox detected exposed private key (.pem file) in the repository
- This may lead to unauthorized system access and full compromise
- AccuKnox proposes the solution to move the key, revoke and regenerate keys.